AI & Deep Learning

ACTL3143 & ACTL5111 Deep Learning for Actuaries

Patrick Laub

UNSW SYDNEY

Lecture Outline

- **Artificial Intelligence**
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- Early Stopping

Different goals of AI

Artificial intelligence describes an agent which is capable of:

Thinking humanly	Thinking rationally
Acting humanly	Acting rationally

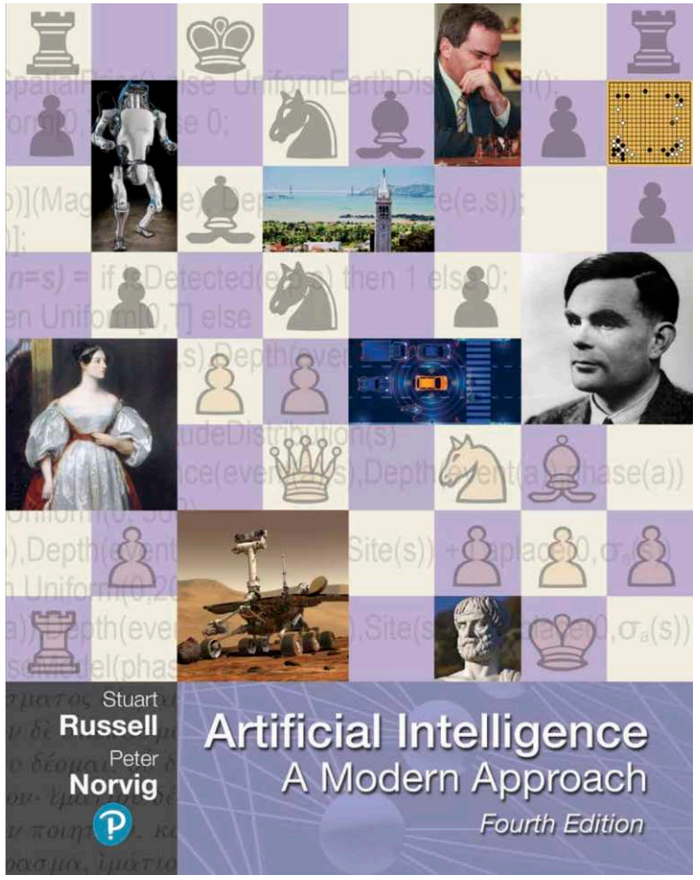
Put another way, what fields are most important to AI?

Cognitive science, psychology	Mathematics, logic and inference
HCI, linguistics and robotics	Computer science, statistics

- Actions are simpler to work with than thought: How do humans even think?
- Acting humanly can be done without intelligence, see ChatGPT
- The focus on actions: delivers results, but in a black-box manner

The rational behaviour paradigm won out

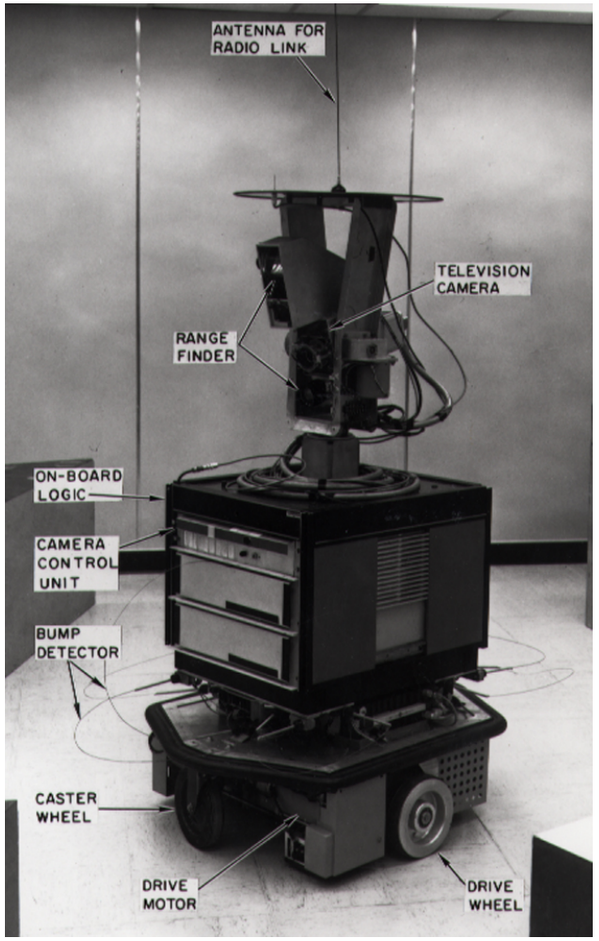
> "For these reasons, the rational-agent approach to AI has prevailed throughout most of the field's history. In the early decades, rational agents were built on logical foundations and formed definite plans to achieve specific goals. Later, methods based on probability theory and machine learning allowed the creation of agents that could make decisions under I uncertainty to attain the best expected outcome. In a nutshell, *AI has focused on the study and construction of agents that* ***do the right thing***."
>
> — Russell & Norvig (2021, p. 22)

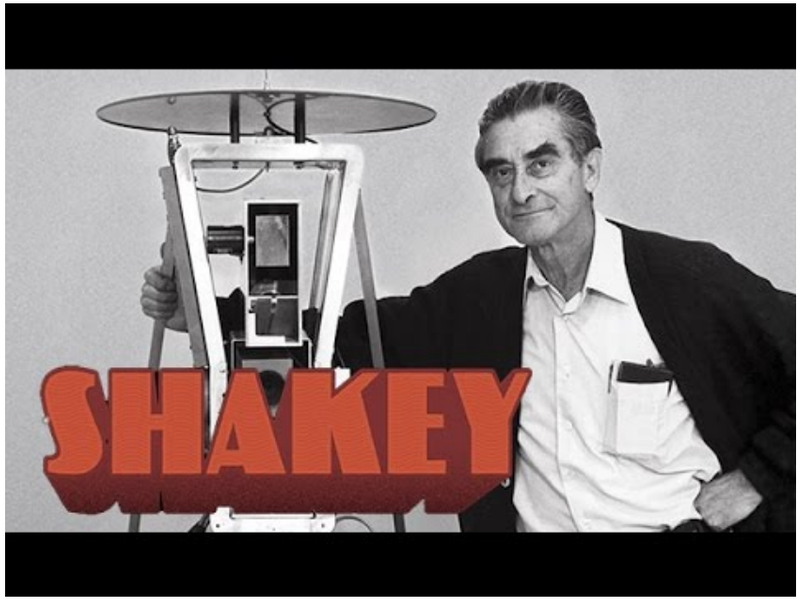


The traditional AI text.

Question: When do you think the term "machine learning" was first coined?

Shakey the Robot (~1966 – 1972)

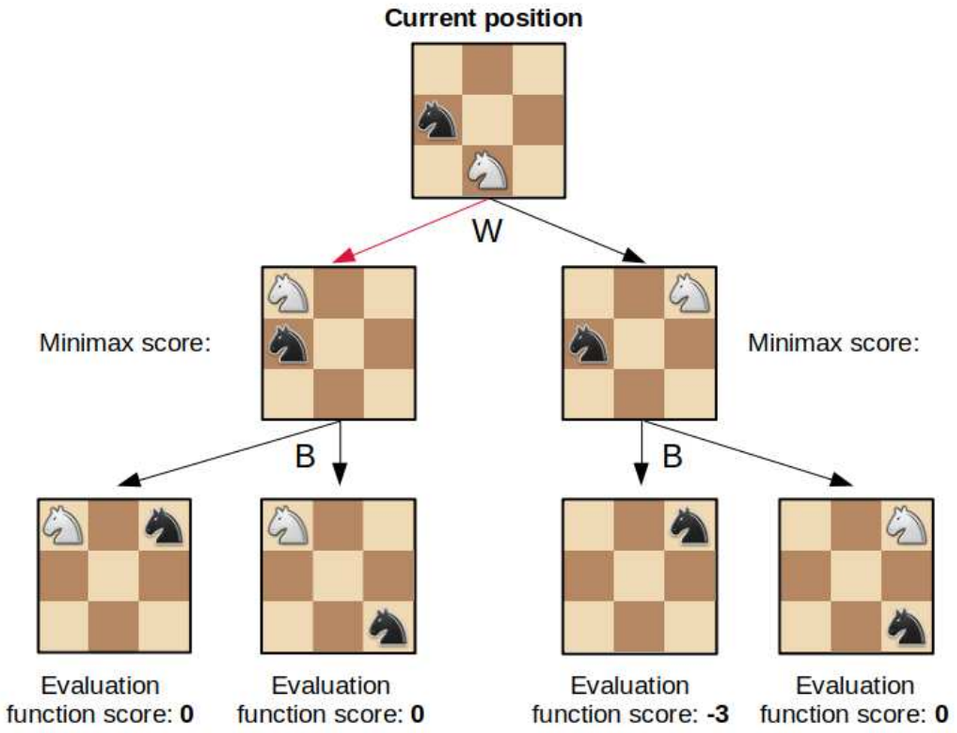




Shakey the Robot

Source: Wikipedia page for the Shakey Project and for the A* search algorithm.

The minimax algorithm



The minimax algorithm for chess.

```
function minimax(position, depth, maximizingPlayer)
   if depth == 0 or game over in position
      return static evaluation of position

   if maximizingPlayer
      maxEval = -infinity
      for each child of position
         eval = minimax(child, depth - 1, false)
         maxEval = max(maxEval, eval)
      return maxEval

   else
      minEval = +infinity
      for each child of position
         eval = minimax(child, depth - 1, true)
         minEval = min(minEval, eval)
      return minEval
```

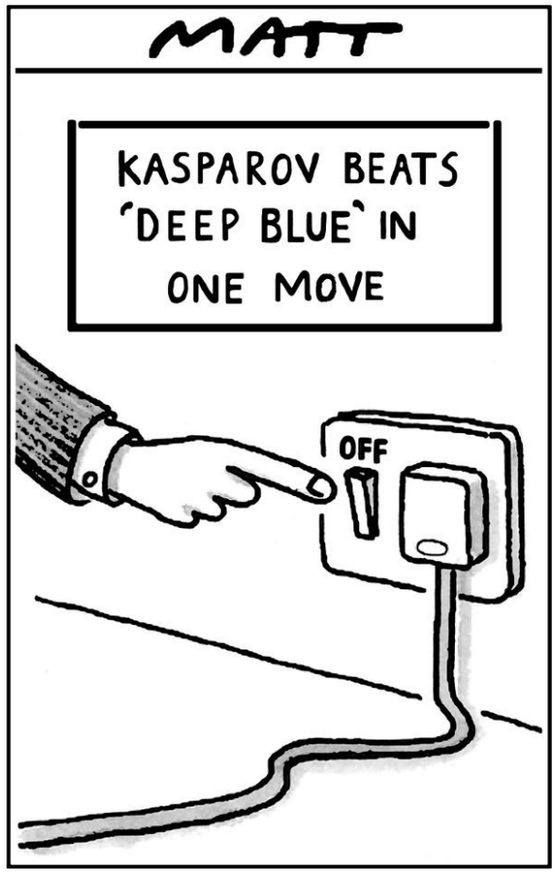
Pseudocode for the minimax algorithm.

Source: codeRtime, Programming a simple minimax chess engine in R, and Sebastian Lague (2018), Algorithms Explained – minimax and alpha-beta pruning

Chess

Deep Blue (1997)

Gary Kasparov playing Deep Blue.



Cartoon of the match.

Sources: Mark Robert Anderson (2017), Twenty years on from Deep Blue vs Kasparov, The Conversation article, and Computer History Museum.

Machine Learning

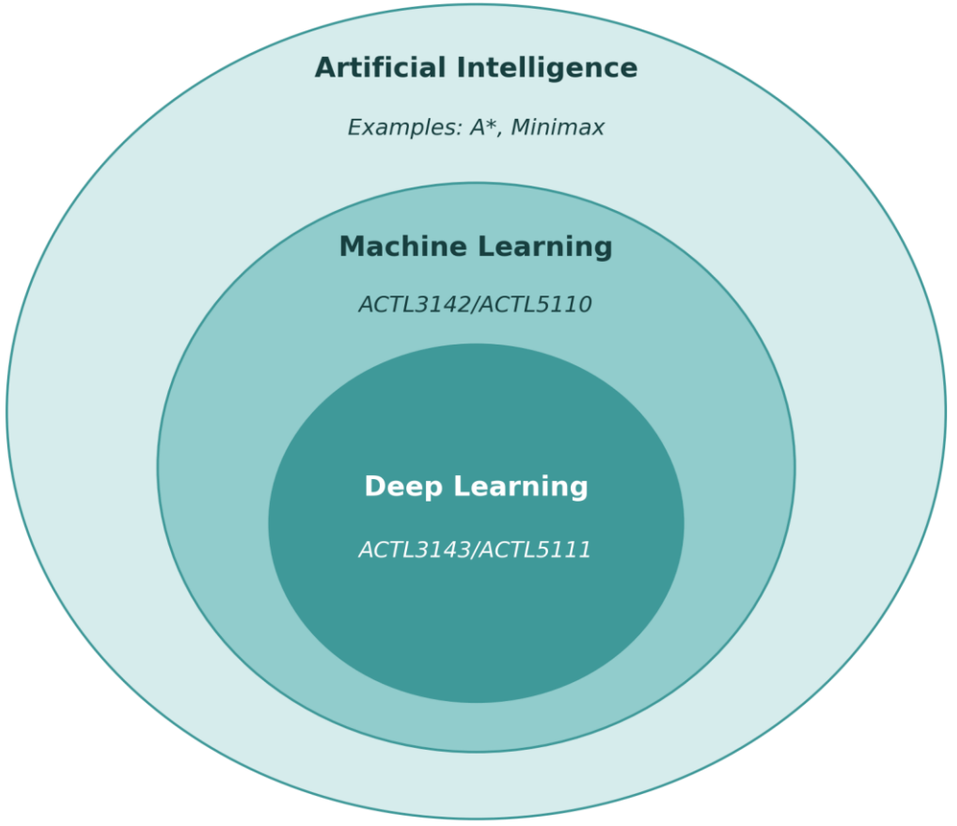
Tried *making a computer smart*, too hard!

Make a computer that can **learn** to be smart.

> "[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed"
>
> — Samuel (1959)

AI eventually become dominated by one approach, called *machine learning*, which itself is now dominated by *deep learning* (neural networks).

You can study a 12 week course on AI and never touch on machine learning...



Artificial Intelligence, Machine Learning, and Deep Learning.

Lecture Outline

- Artificial Intelligence
- **Deep Learning Successes**
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- Early Stopping

Image Classification I

What is this?

Options:

1. punching bag
2. goblet
3. red wine
4. hourglass
5. balloon

Source: Wikipedia

Image Classification II

What is this?

Options:

1. sea urchin
2. porcupine
3. echidna
4. platypus
5. quill

Source: Wikipedia

Image Classification III

What is this?

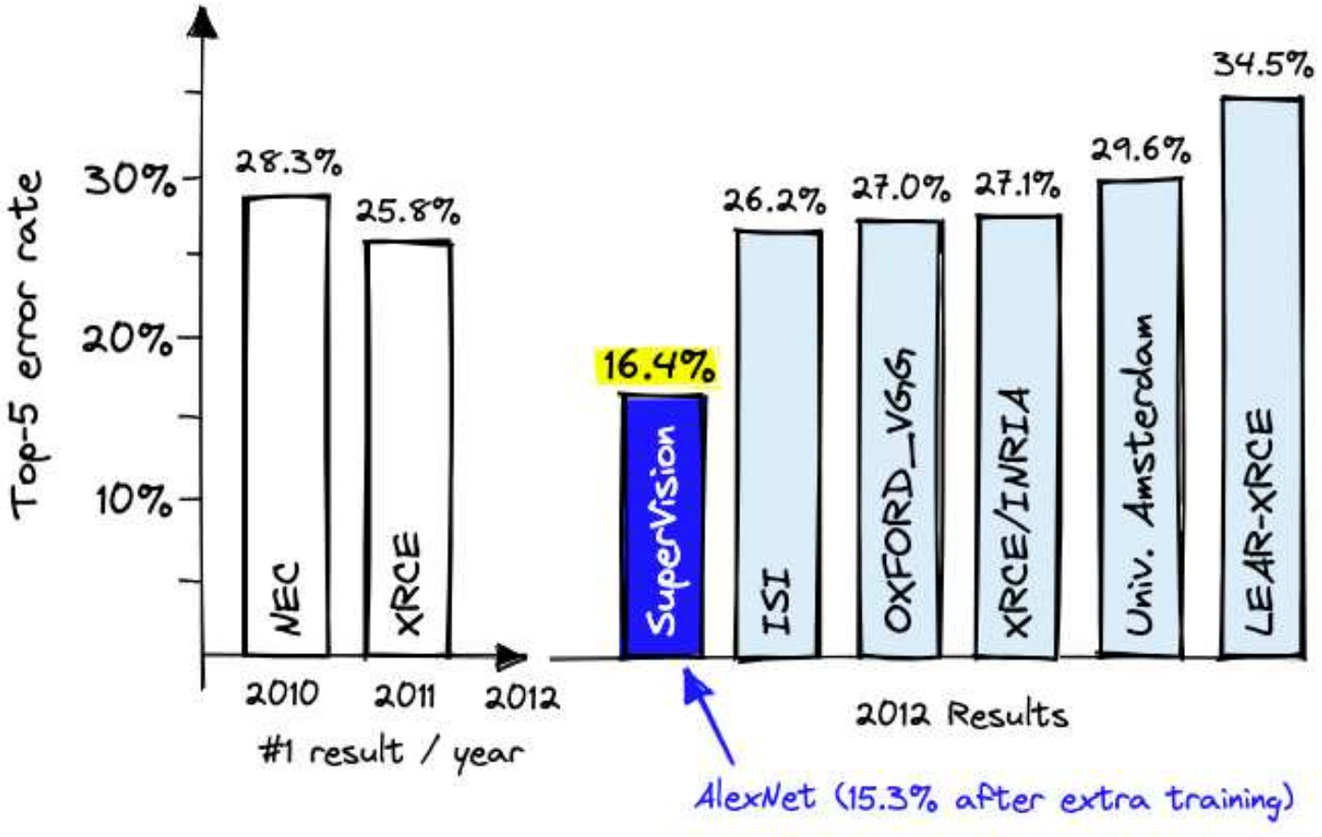
Options:

1. dingo
2. malinois
3. German shepherd
4. muzzle
5. kelpie

Source: Wikipedia

ImageNet Challenge (2012)

ImageNet and the *ImageNet Large Scale Visual Recognition Challenge (ILSVRC)*; originally 1,000 synsets.

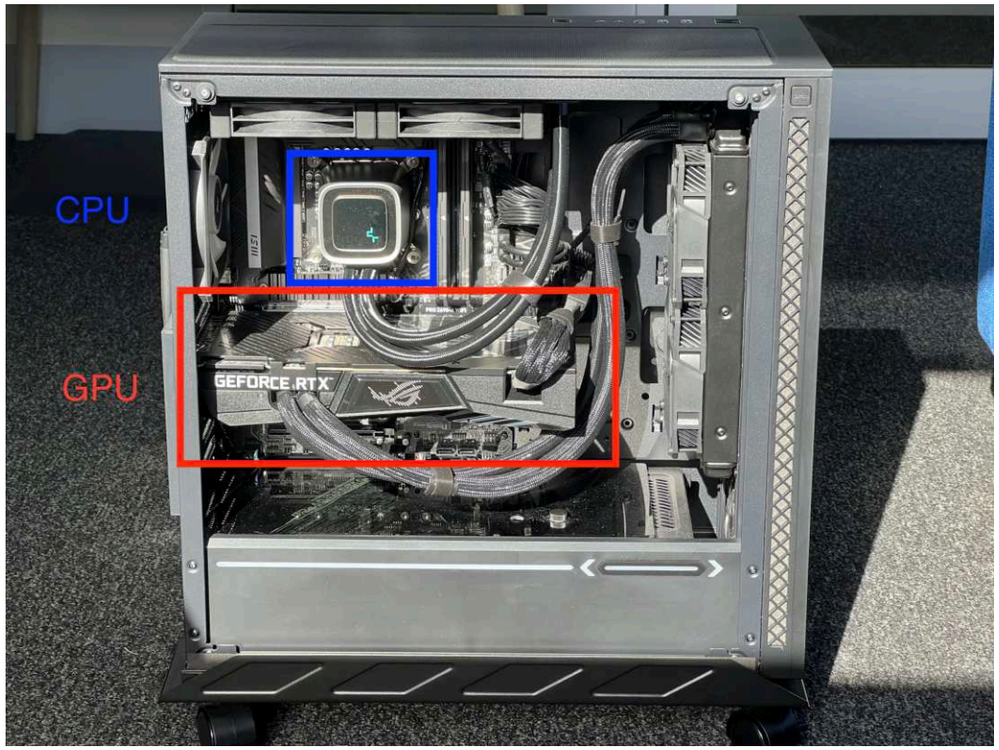


AlexNet — a neural network developed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton — won the ILSVRC 2012 challenge convincingly.

Source: James Briggs & Laura Carnevali, *AlexNet and ImageNet: The Birth of Deep Learning*, Embedding Methods for Image Search, Pinecone Blog

Needed a graphics card

A graphics processing unit (GPU)



A PC with the GPU and CPU marked in red and blue.

> "**4.2. Training on multiple GPUs** A single GTX 580 GPU has only 3GB of memory, which limits the maximum size of the networks that can be trained on it. It turns out that 1.2 million training examples are enough to train networks which are too big to fit on one GPU. Therefore we spread the net across two GPUs."
>
> — Krizhevsky et al. (2012)

Lee Sedol plays AlphaGo (2016)

Deep Blue was a win for AI, AlphaGo a win for DL.

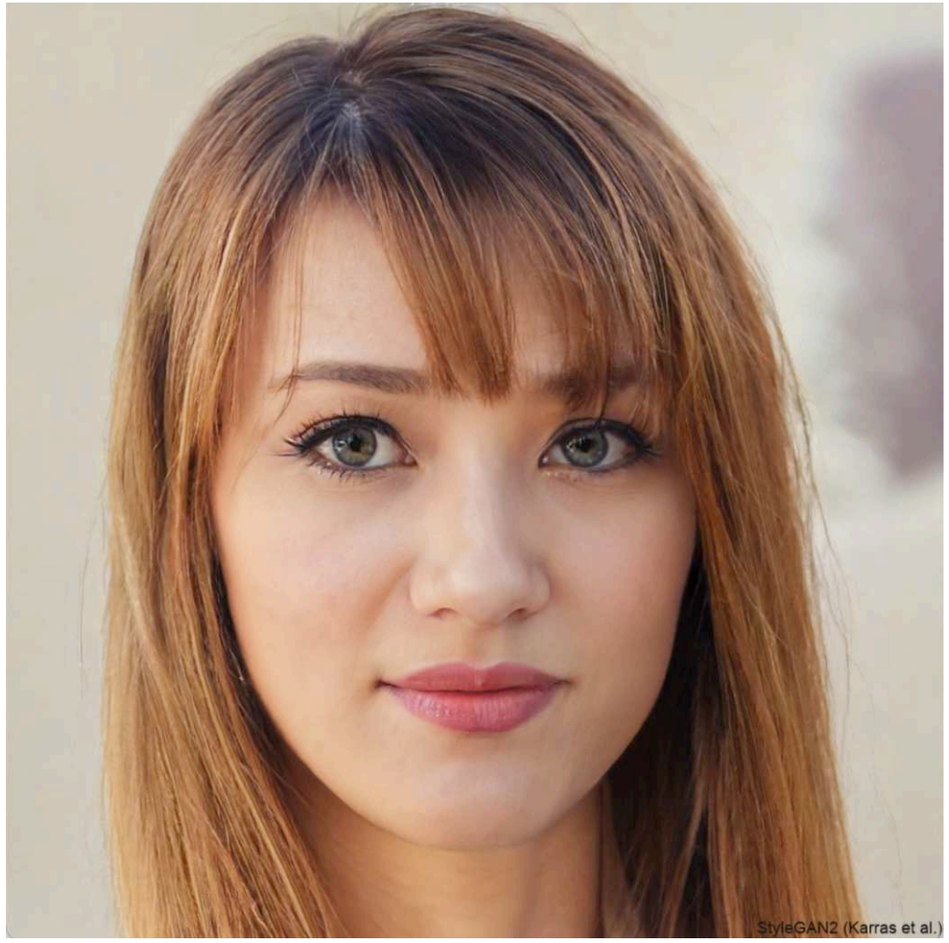
Lee Sedol playing AlphaGo AI

I highly recommend this documentary about the event.

Source: Patrick House (2016), AlphaGo, Lee Sedol, and the Reassuring Future of Humans and Machines, New Yorker article.

Generative Adversarial Networks (2014)

https://thispersondoesnotexist.com/



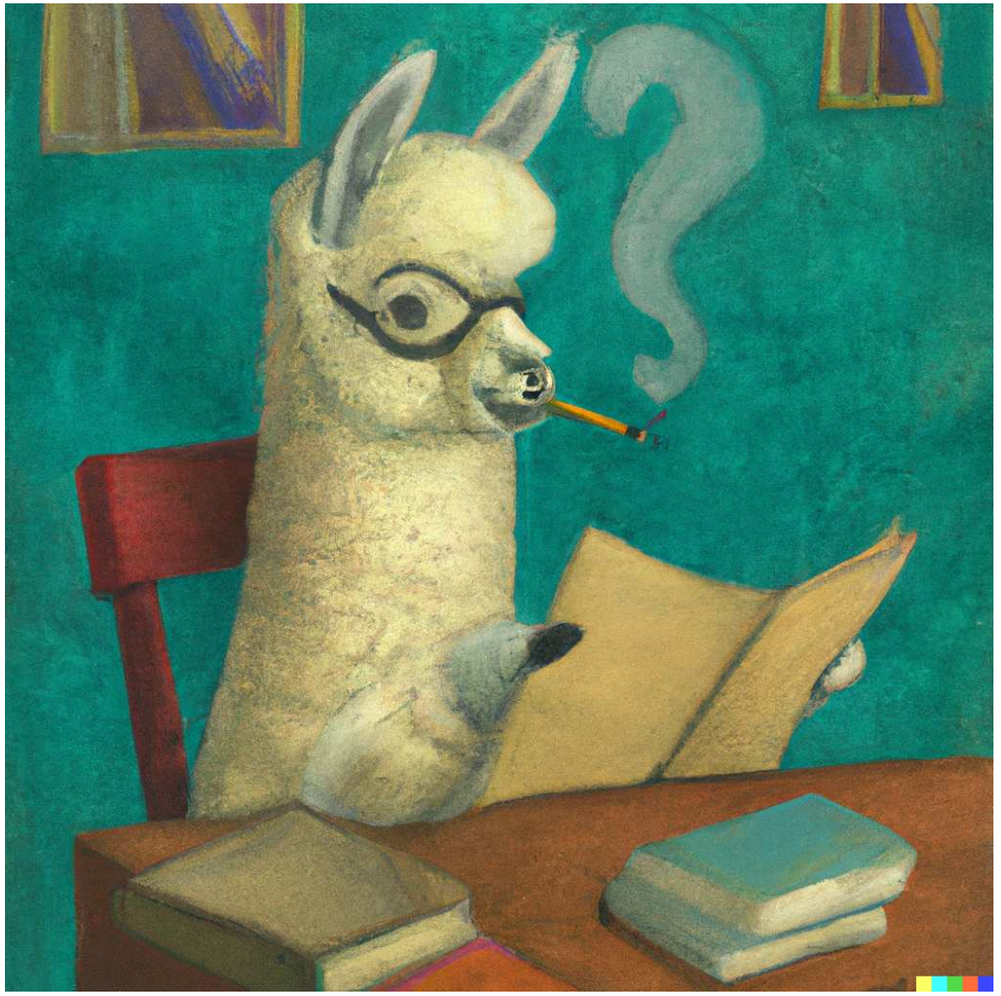
A GAN-generated face

A GAN-generated face

Source: Goodfellow et al. (2014).

Diffusion models (2020)

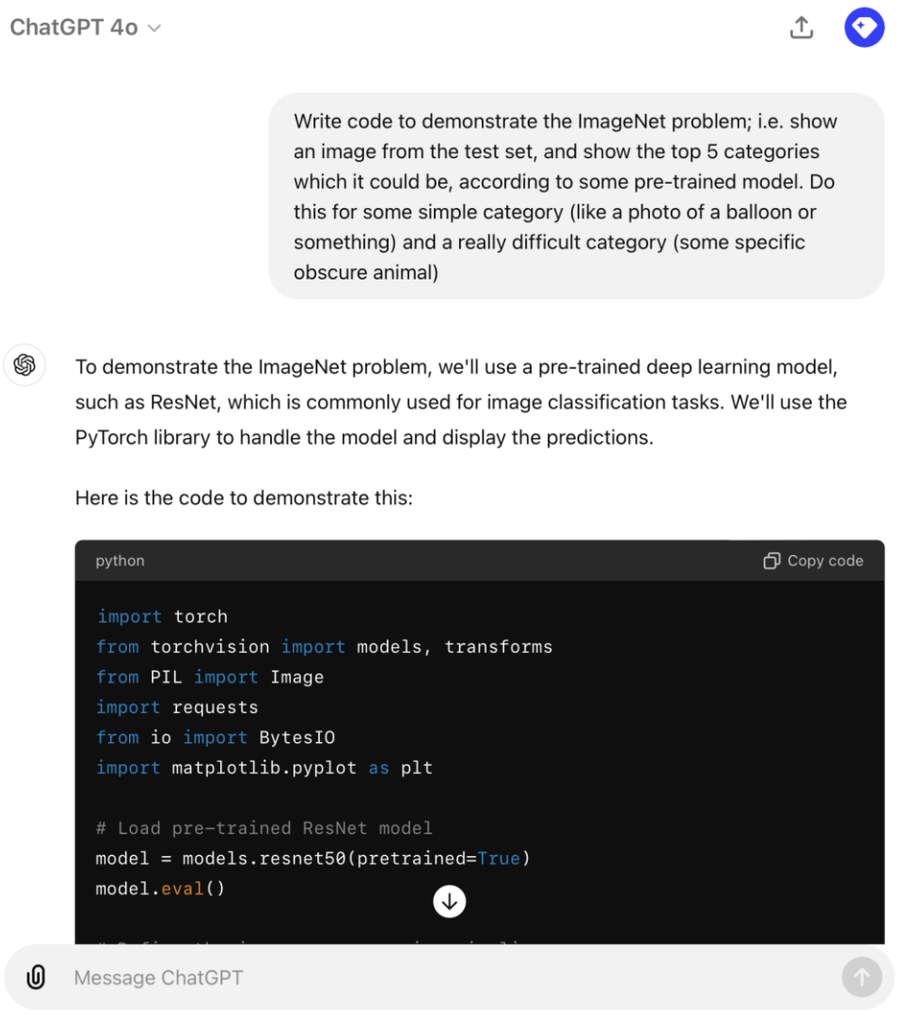
Painting of avocado skating while wearing a hoodie



A surrealist painting of an alpaca studying for an exam

Source: Ho et al. (2020). Images generated with Dall-E 2, prompts by ACTL3143 students in 2022.

ChatGPT (2022)



AI predictions in the ImageNet demo were from ChatGPT code.

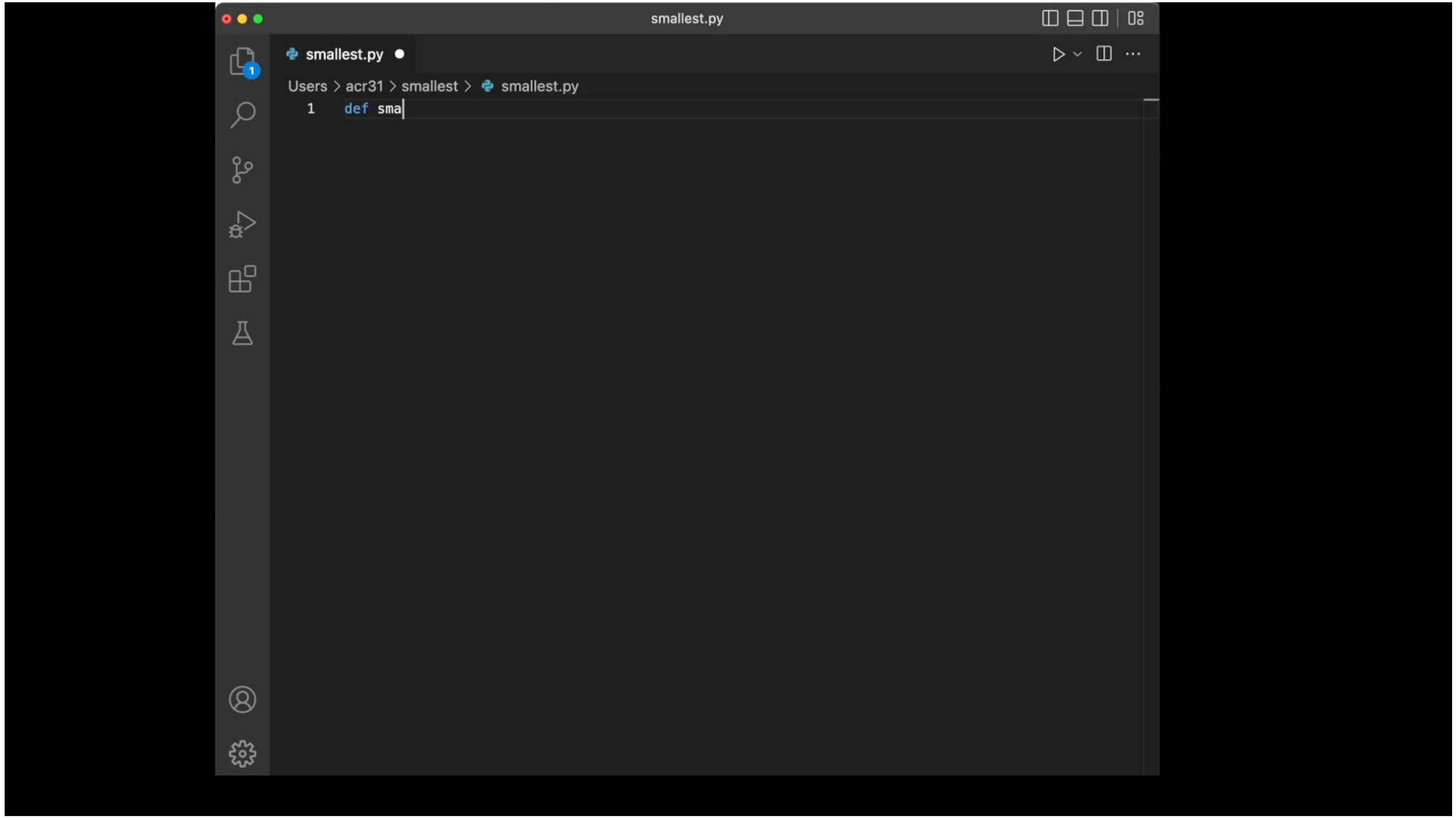
Test ChatGPT's ability to:

- generate images
- translate code
- explain code
- run code
- analyse a dataset
- critique code
- critique writing
- voice chat with you

Compare to Copilot.

UNSW SYDNEY

GitHub Copilot (2022)

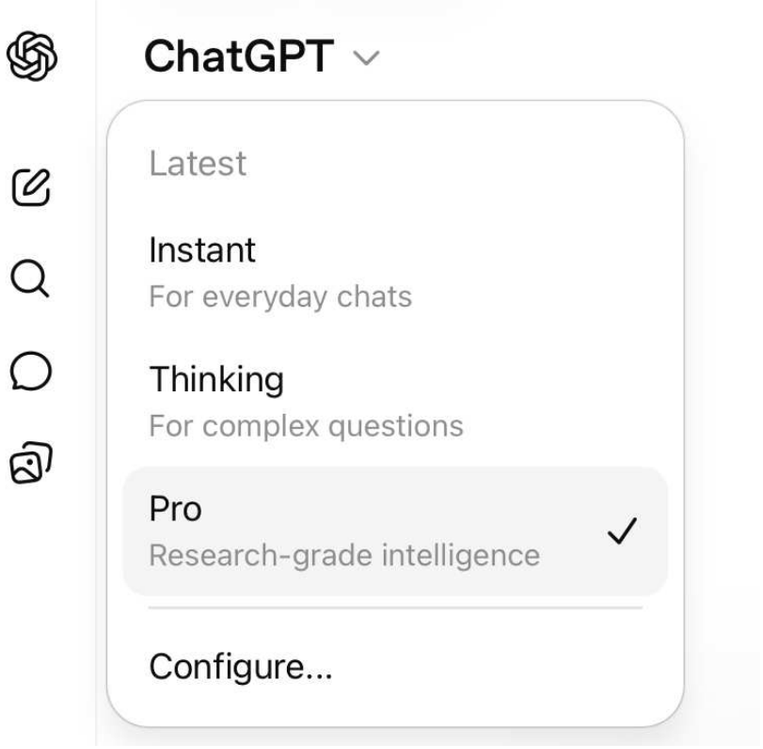


You can get extra usage for free with GitHub Education for Students

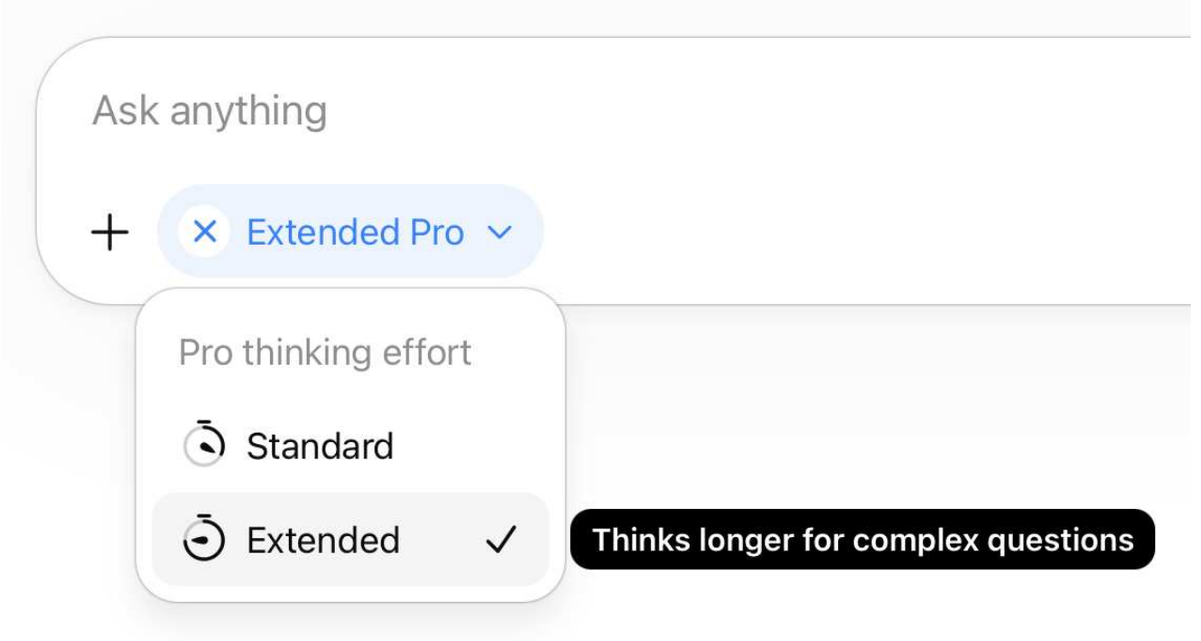
Source: GitHub Blog

Reasoning models (2024)

Reasoning is basically automating the old trick of adding “do this step-by-step” to your prompts.



Choosing the Pro model.



Setting the thinking effort to “Extended”.

Using the larger language models

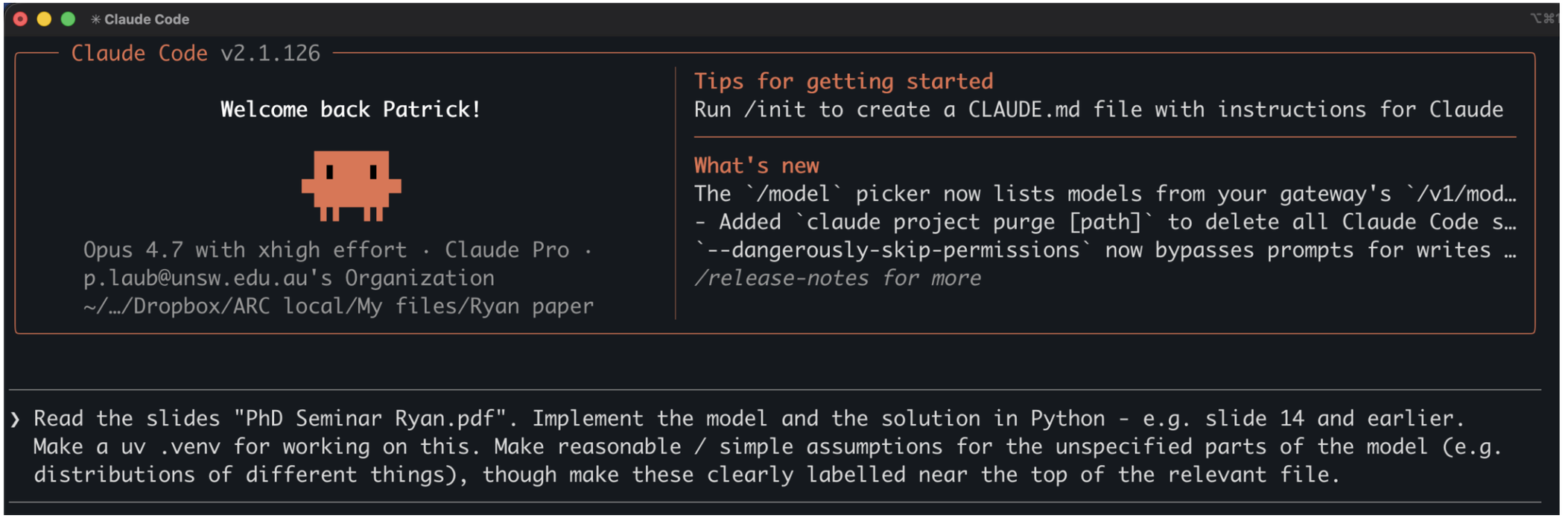
- Zoom out as far as possible
- Give it as much relevant context as possible
- Better if it's easy to ingest (LaTeX or Python/R code), otherwise it has to convert to text
- A simple instruction in the prompt is enough
- Context sizes for the top models have become quite long
- I've had the best results when it reasons for **20–30 minutes**; then I review each potential issue it finds.

Give me a list of issues (maths, logic, spelling, grammar) in the following manuscript.

```
"""\documentclass[
  journal=aas,
  manuscript=article-type,
  year=2025,
  volume=37,
]{cup-journal}

\usepackage{amsfonts}
\usepackage{amsmath}
\usepackage{amsthm}
\usepackage{graphicx}
\usepackage{subfig}
\usepackage{caption}
\usepackage{xcolor}
```

A typical prompt I use.

Claude Code (2025)



Let the LLM take control of your terminal/computer.

To get an introduction to using the terminal, check this recording.

Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- **Types of Machine Learning Tasks**
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- Early Stopping

Predictive versus generative

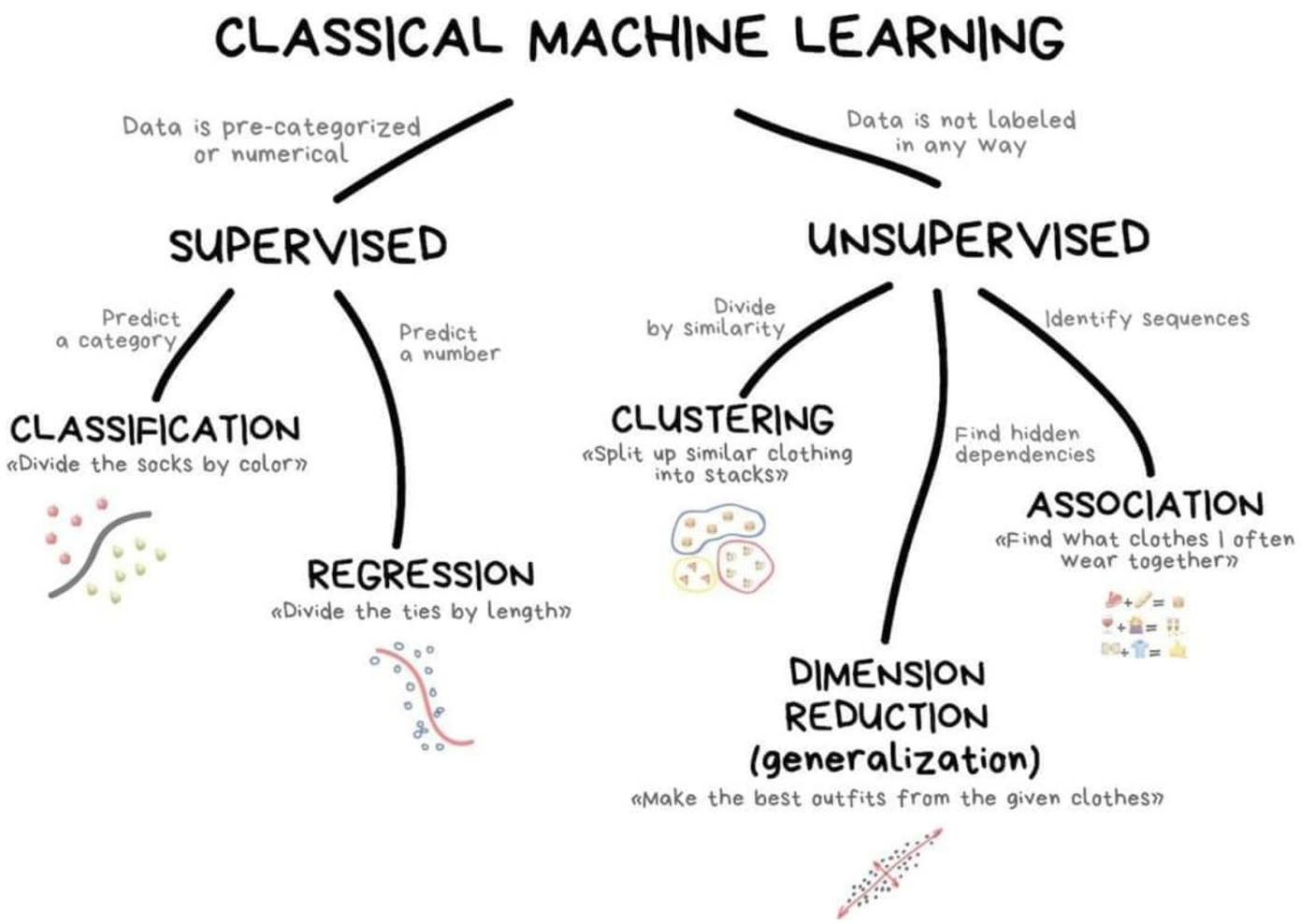
We focus on predictive not generative AI in this course.

Our school has two new courses starting in 2026:

- **ACTL4307** “Generative AI for Actuaries”
- **ACTL4306** “Quantitative Ethical AI for Risk & Actuarial Applications”

This course focuses on **neural networks for supervised learning**, and these techniques are the fundamental building blocks for all the others parts of modern AI.

A taxonomy of problems



New ones:

- Reinforcement learning
- Semi-supervised learning
- Active learning

Machine learning categories in ACTL3142.

Source: Kaggle, Getting Started.

Examples of supervised learning

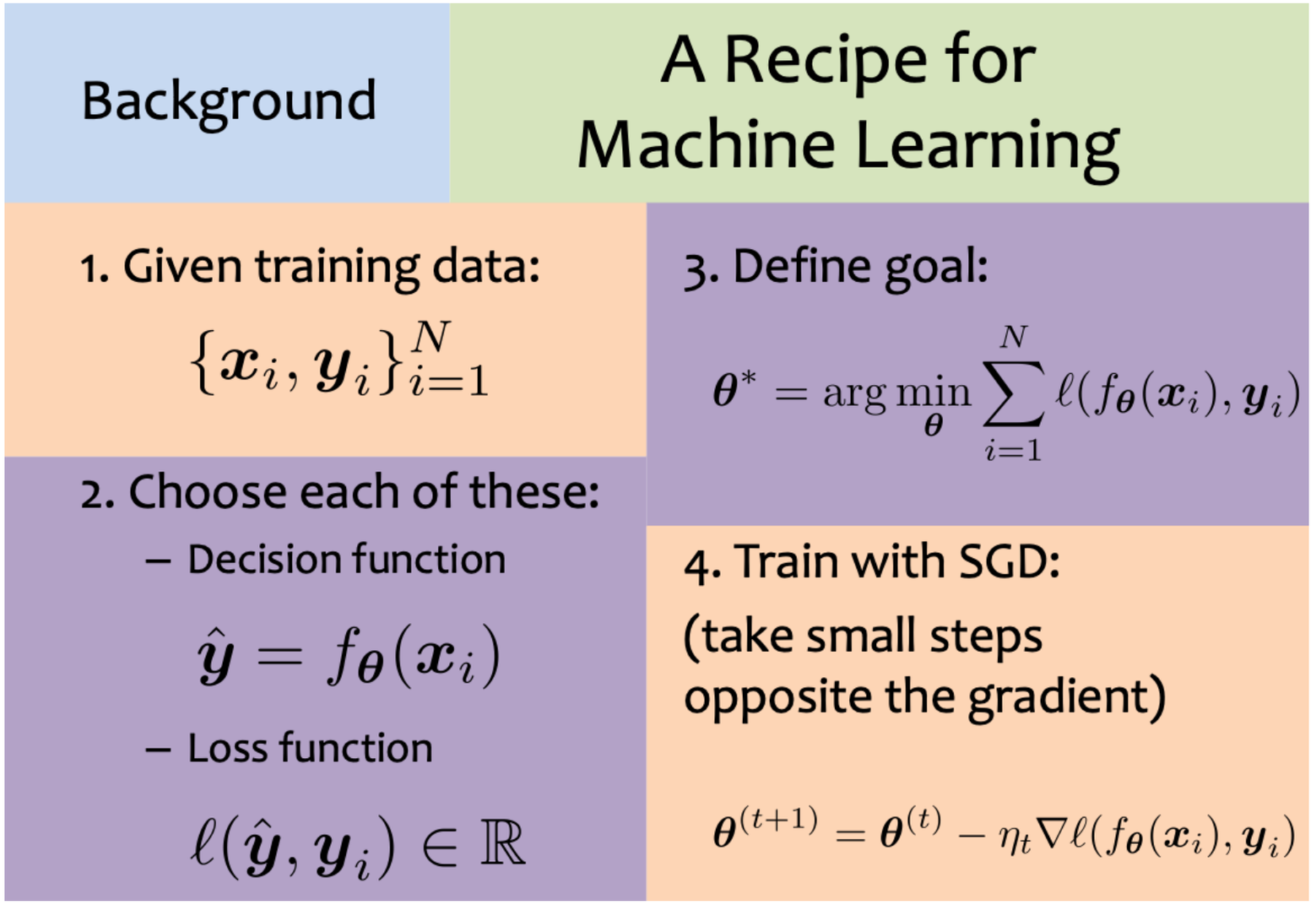
Regression:

- Given policy $\hookrightarrow$ predict the rate of claims.
- Given policy $\hookrightarrow$ predict claim severity.
- Given a reserving triangle $\hookrightarrow$ predict future claims.

Classification:

- Given a claim $\hookrightarrow$ classify as fraudulent or not.
- Given a customer $\hookrightarrow$ predict customer retention patterns.

Supervised learning: mathematically

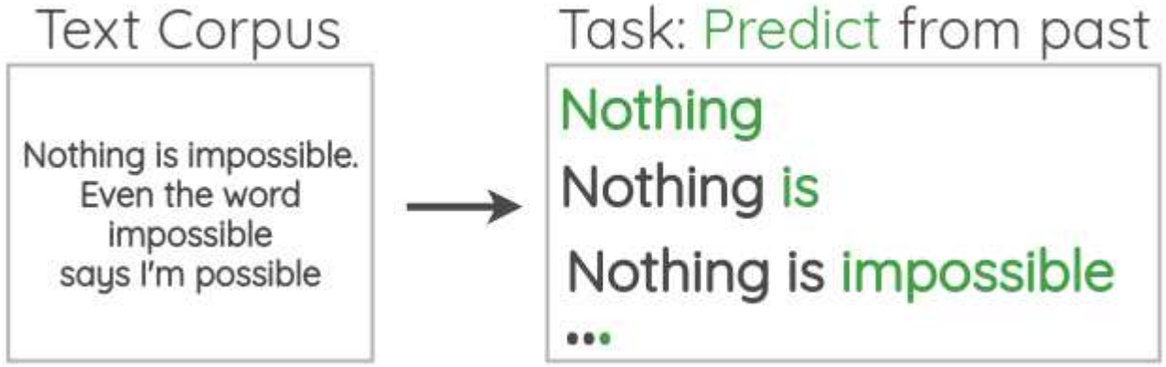


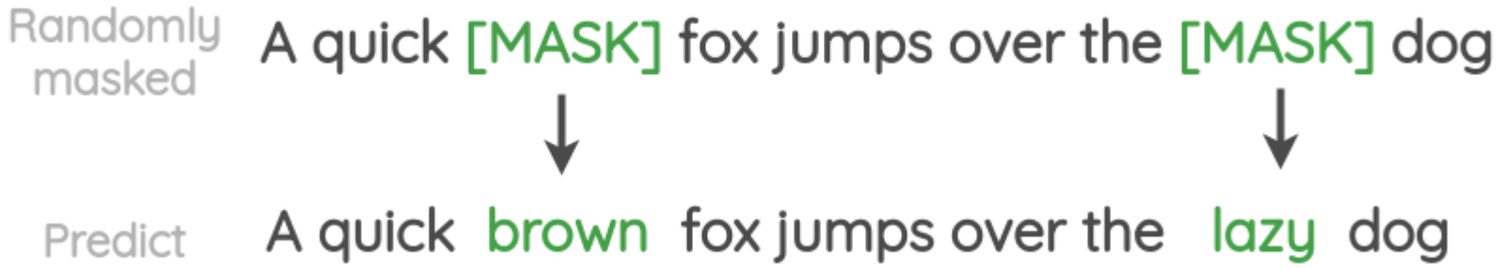
A recipe for supervised learning.

Source: Matthew Gormley (2021), Introduction to Machine Learning Lecture Slides, Slide 67.

Self-supervised learning

Data which 'labels itself'. Example: language model.





'Autoregressive' (e.g. GPT) versus 'masked' model (e.g. BERT).

Source: Amit Chaudhary (2020), Self Supervised Representation Learning in NLP.

Example: image super-resolution

Original image: it is now the target

Downscaled to a lower resolution: it is now the input

Other examples: image inpainting, denoising images.

Test image: Eileen Collins (NASA, public domain).

Example: Deoldify images

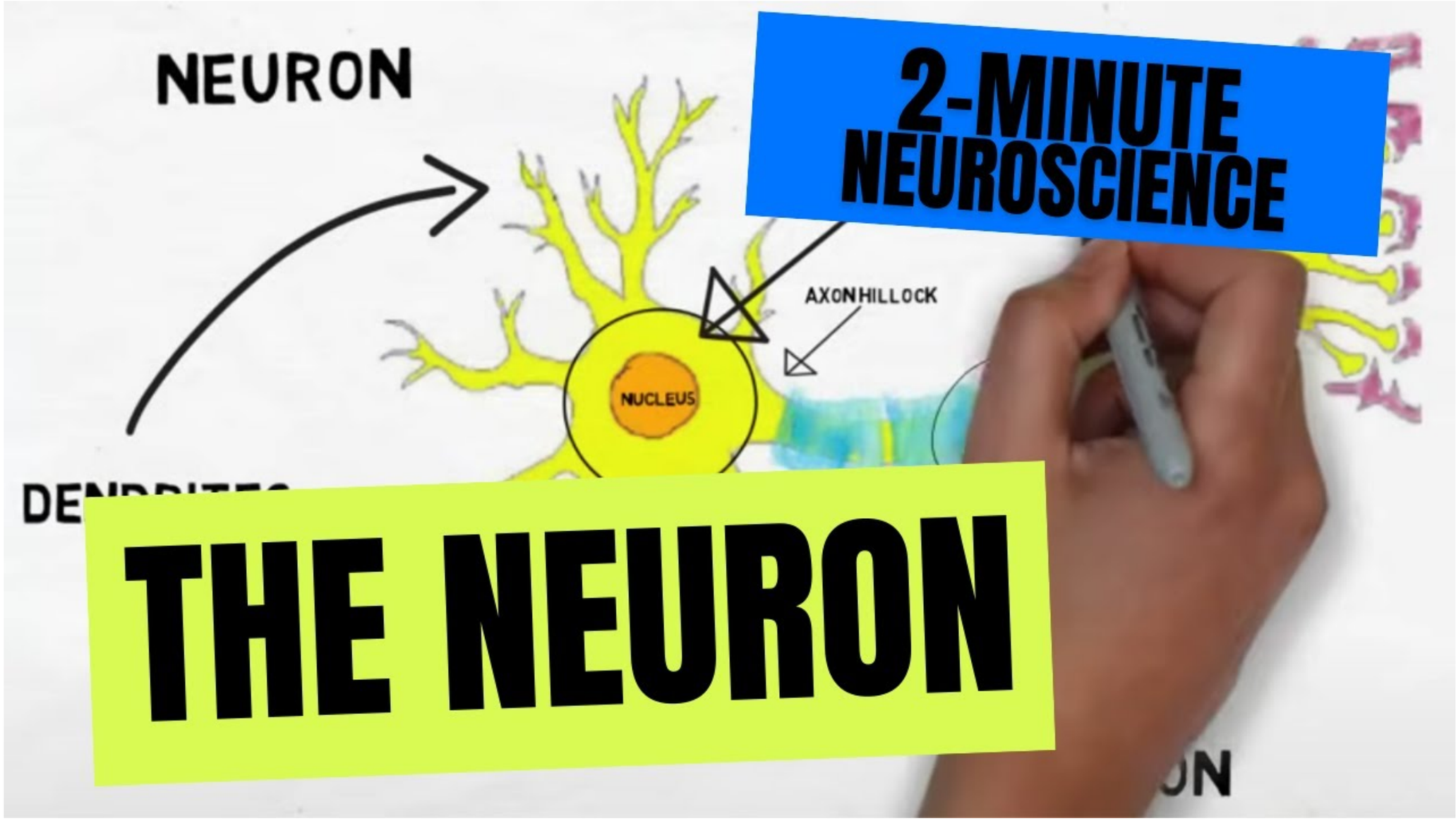
A deoldified version of the famous “Migrant Mother” photograph.

Source: Deoldify package.

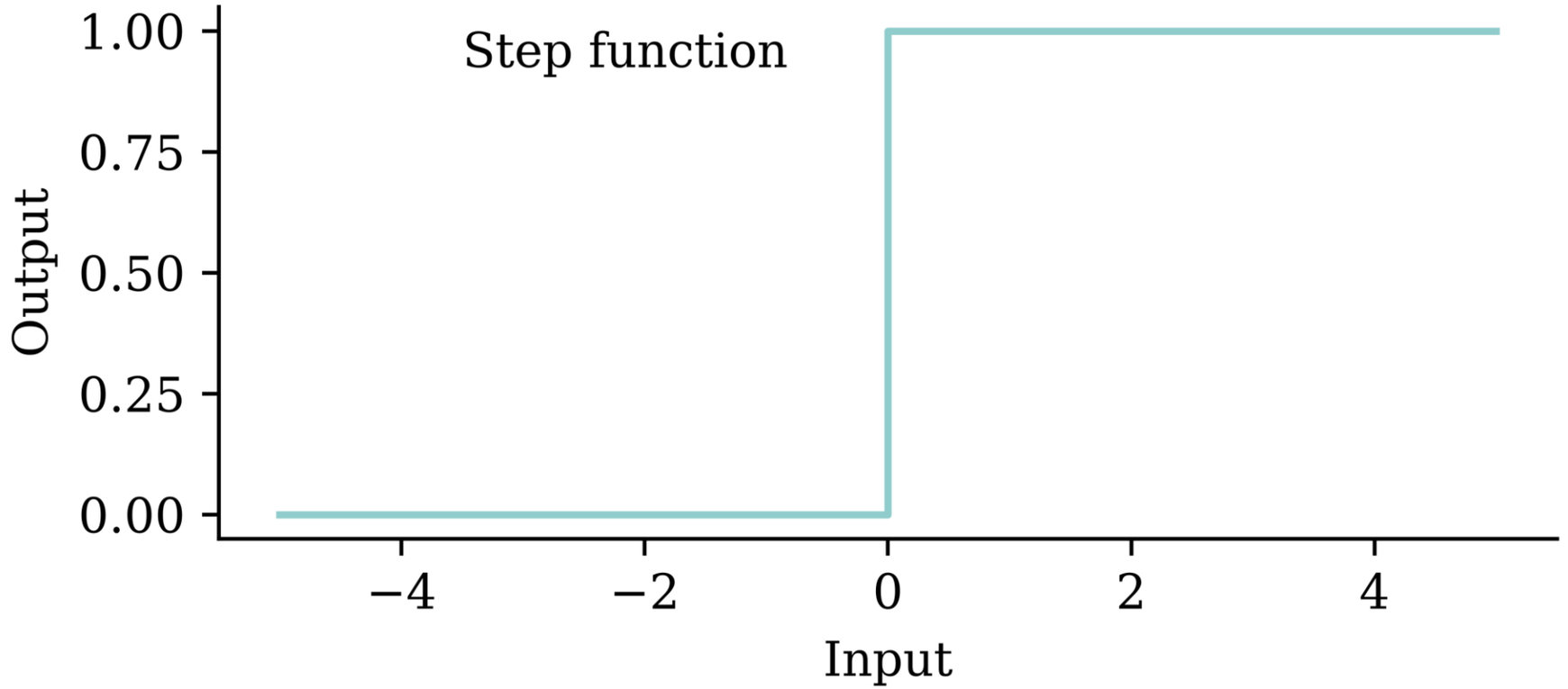
Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- **Neural Networks**
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- Early Stopping

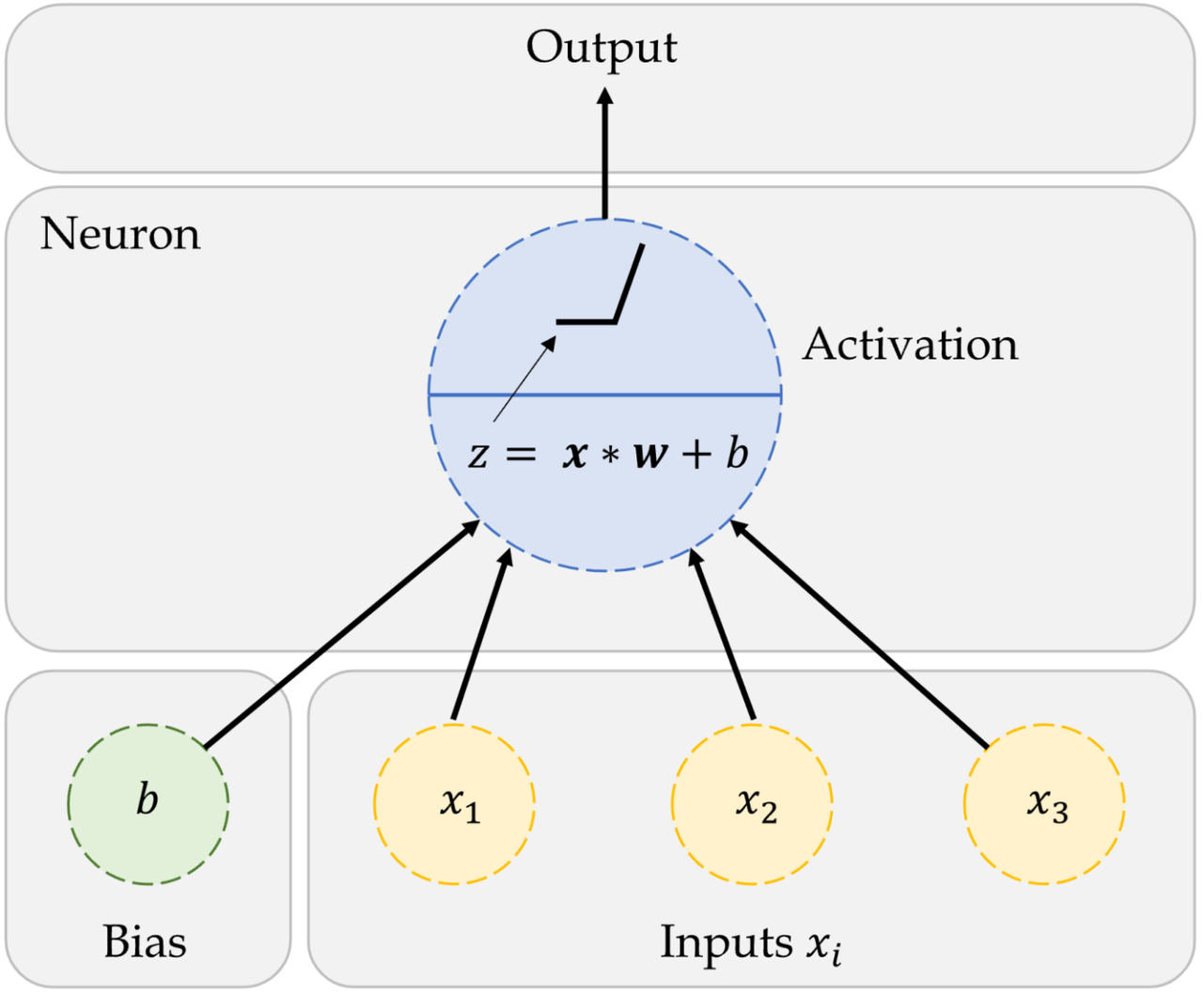
How do real neurons work?



A neuron ‘firing’



An artificial neuron



A neuron in a neural network with a ReLU activation.

Source: Marcus Lautier (2022).

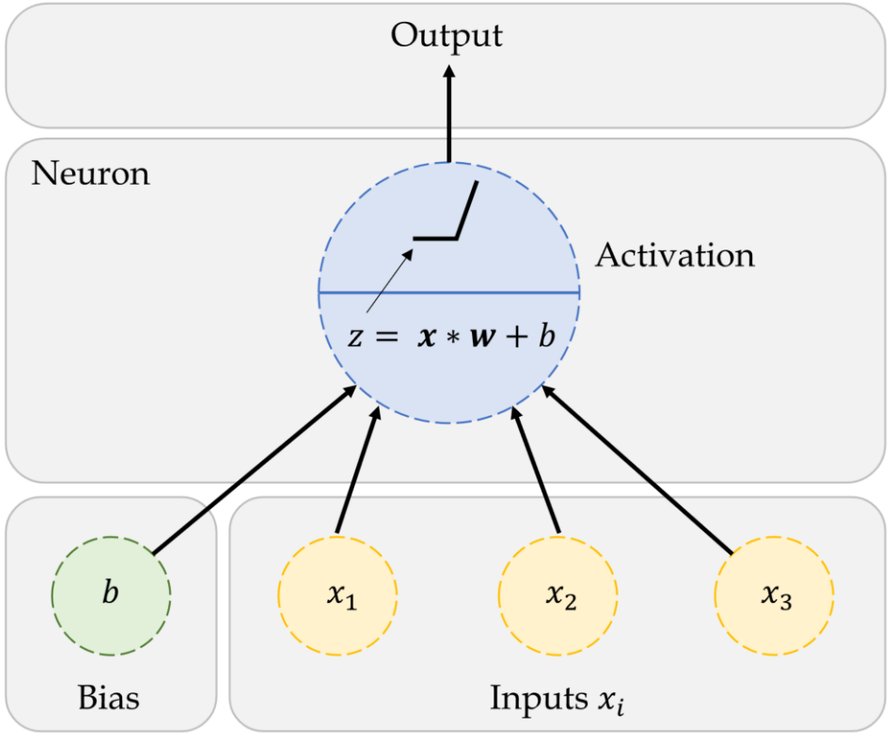
One neuron

$$
\begin{aligned}
z &= x_1 \times w_1 + \\
&\quad x_2 \times w_2 + \\
&\quad x_3 \times w_3.
\end{aligned}
$$

$$
a = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}
$$

Here, x_1, x_2, x_3 are just some fixed data.

The weights w_1, w_2, w_3 should be 'learned'.



A neuron in a neural network with a ReLU activation.

Source: Marcus Lautier (2022).

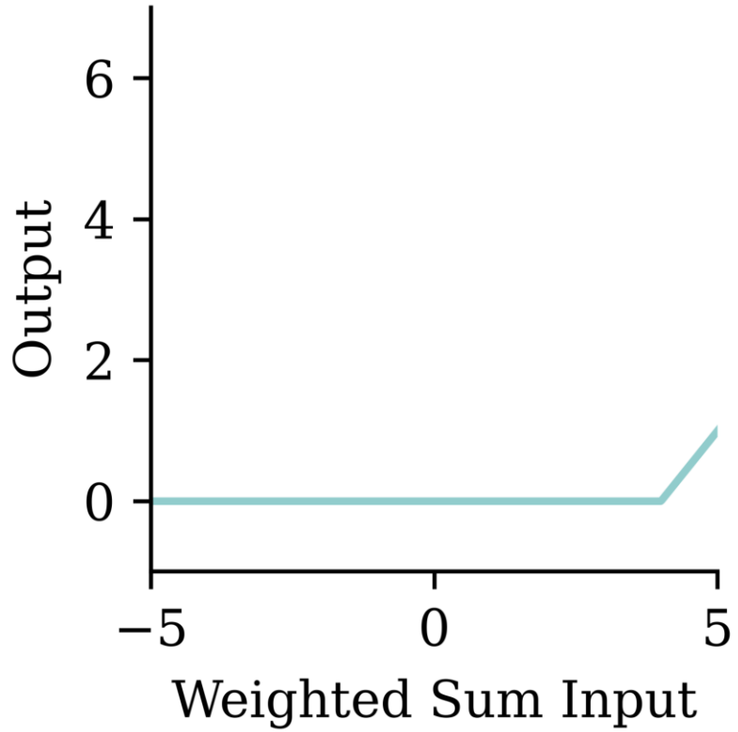
One neuron with bias

$$
\begin{aligned}
z &= x_1 \times w_1 + \\
&\quad x_2 \times w_2 + \\
&\quad x_3 \times w_3 + b.
\end{aligned}
$$

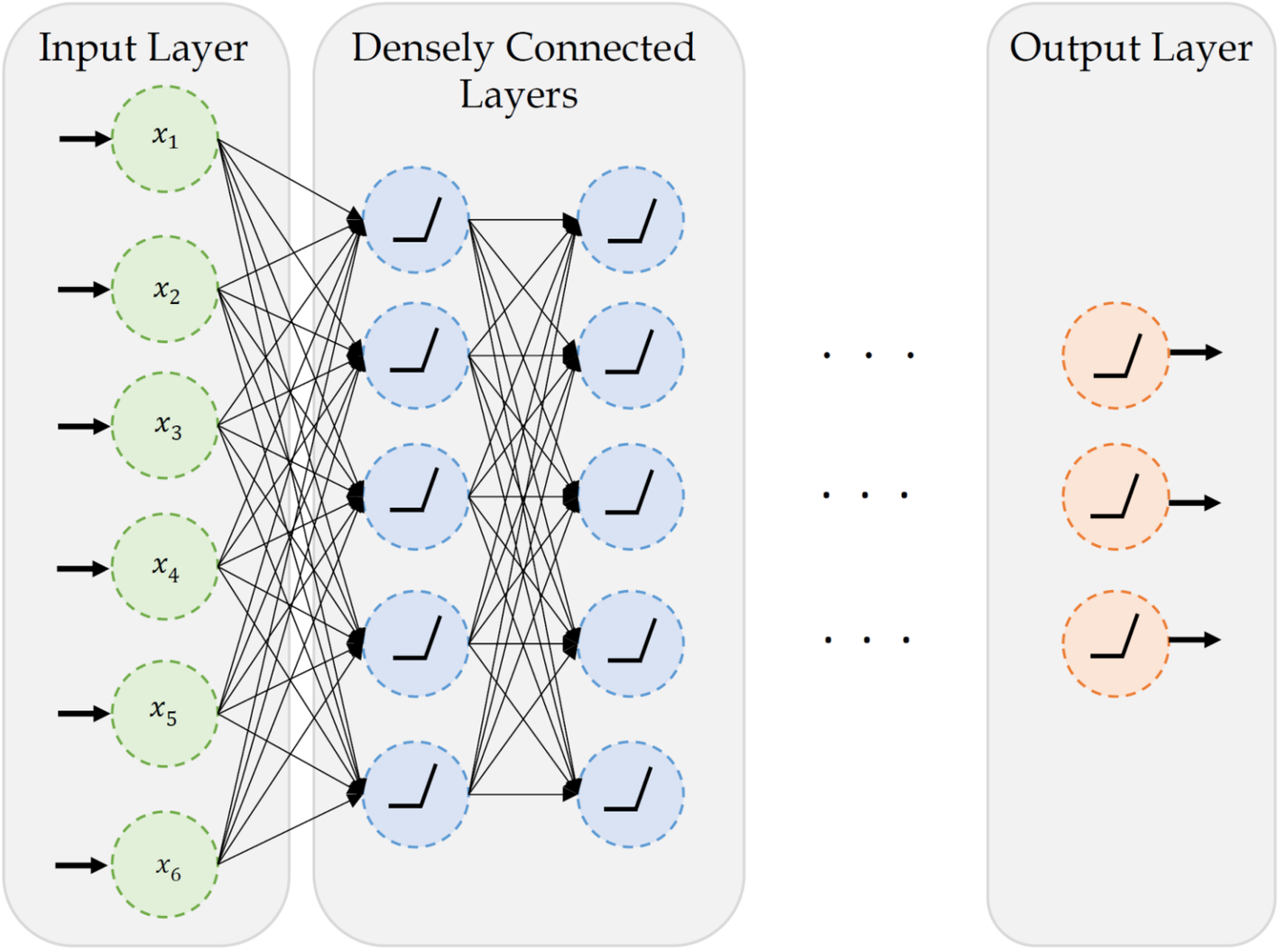
$$
a = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}
$$

The weights w_1, w_2, w_3 and bias b should be 'learned'.

Bias = -4 | 0 | 4



A basic neural network



A basic fully-connected/dense network.

Source: Marcus Lautier (2022).

Step-function activation

Perceptrons

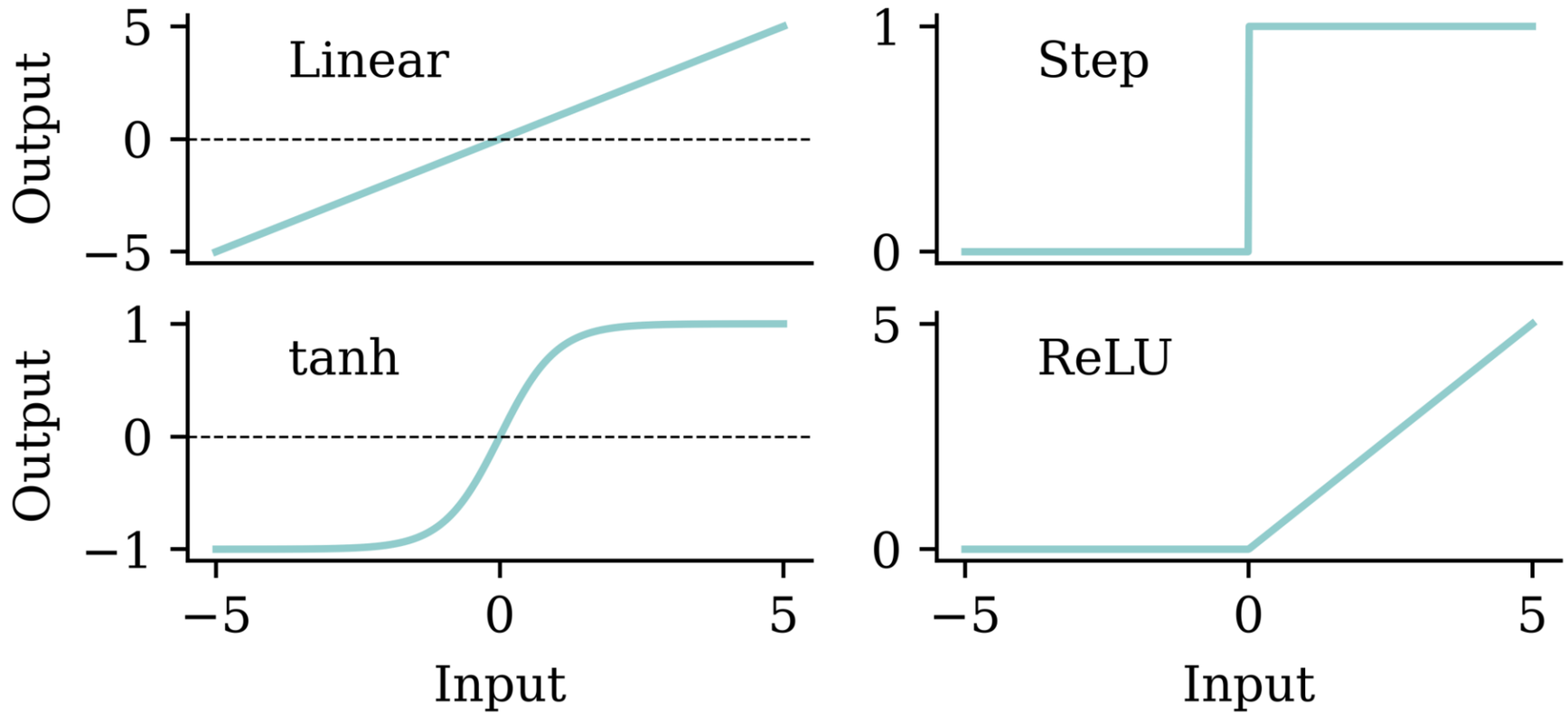
Brains and computers are binary, so make a perceptron with binary data. Seemed reasonable, impossible to train.

Modern neural network

Replace binary state with continuous state. Still rather slow to train.

> **Note**
>
> It's a neur**al** network made of neur**on**s, not a "neuron network".

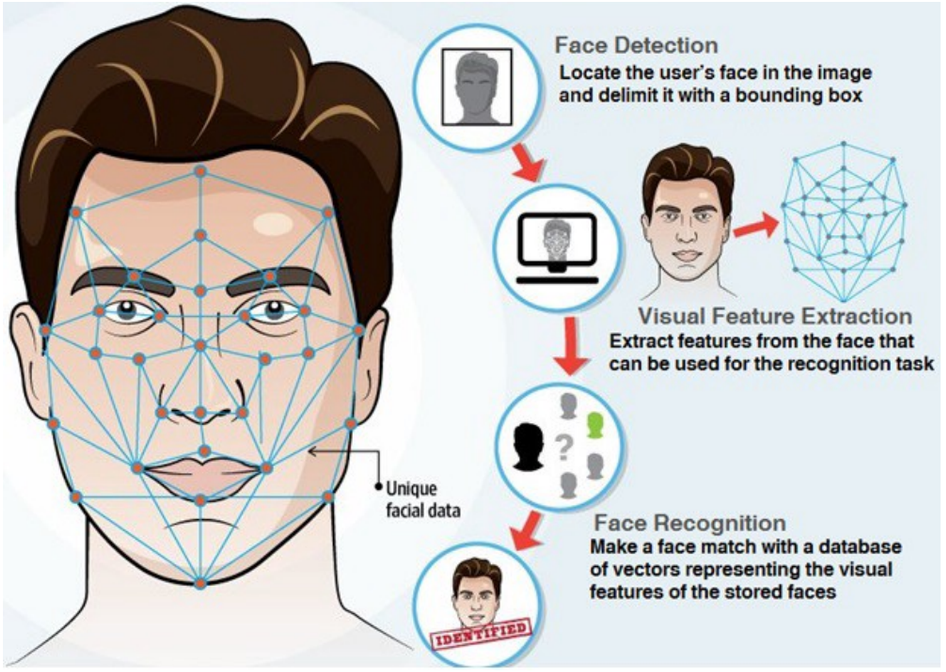
Try different activation functions

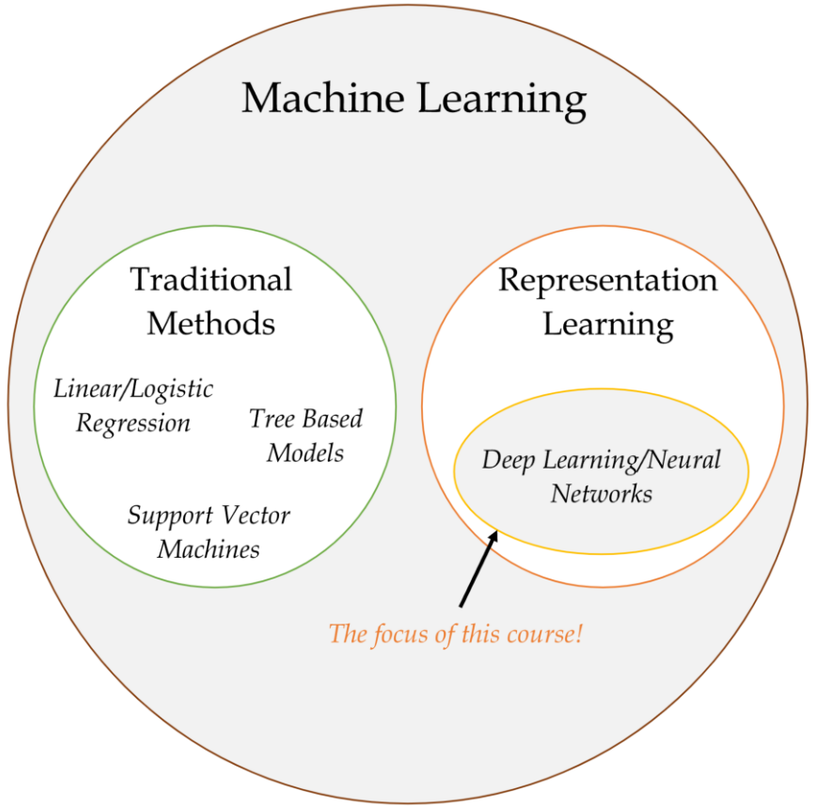


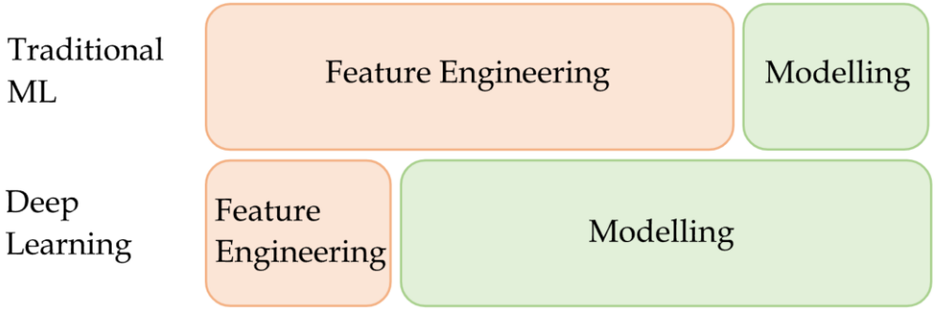
Flexible

> One can show that an MLP is a **universal approximator**, meaning it can model any suitably smooth function, given enough hidden units, to any desired level of accuracy (Hornik 1991). One can either make the model be “wide” or “deep”; the latter has some advantages...
>
> — Murphy (2012, p. 566)

Feature engineering







Neural networks can learn how to manipulate the inputs (with enough data). That doesn't mean deep learning is always the best option!

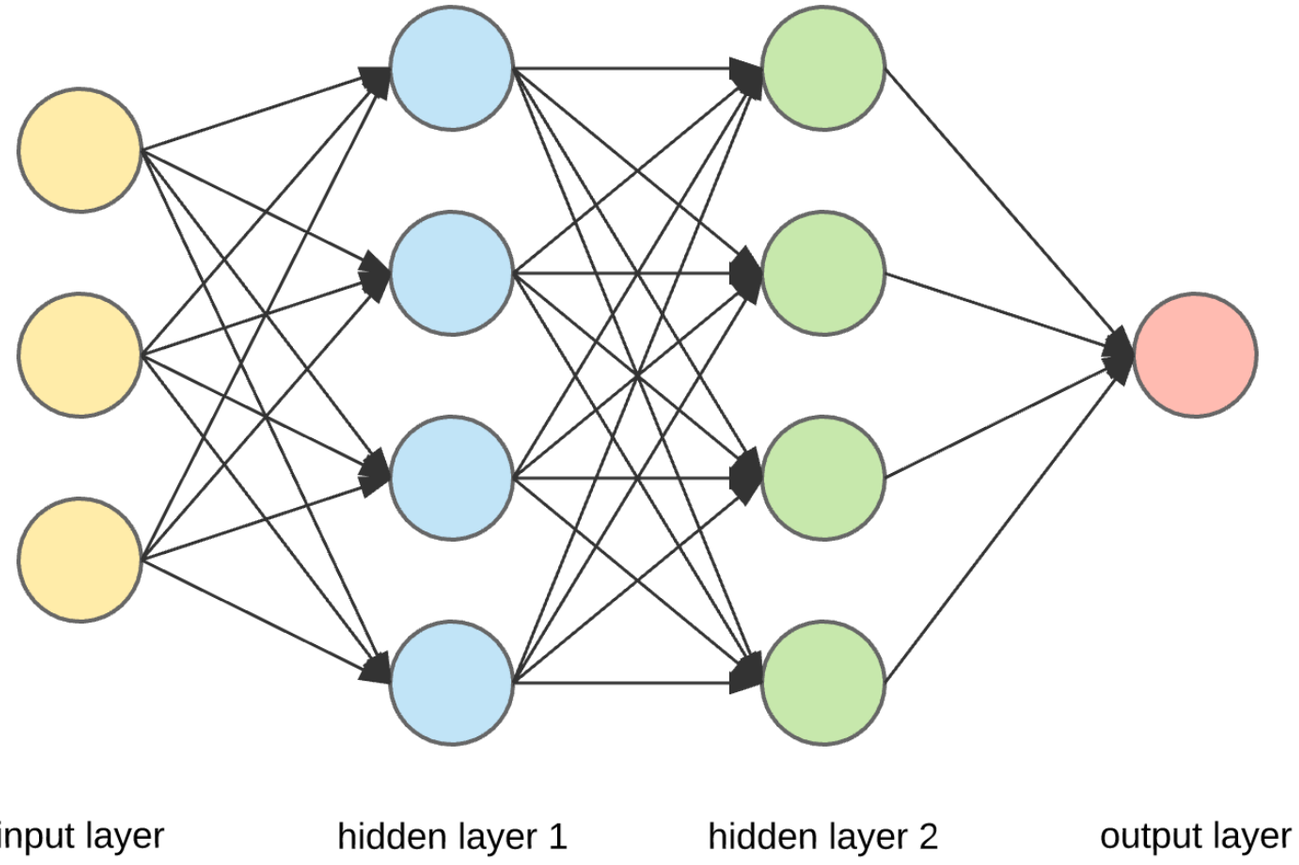
Sources: Marcus Lautier (2022) & Fenjiro (2019), *Face Id: Deep Learning for Face Recognition*, Medium.

Quiz

In this ANN, how many of the following are there:

- features,
- targets,
- weights,
- biases, and
- parameters?

What is the depth?



An artificial neural network.

Source: Dertat (2017), *Applied Deep Learning - Part 1: Artificial Neural Networks*, Medium.

Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- **California House Price Prediction**
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- Early Stopping

Imports needed for this demo

```
import random

import numpy as np
import pandas as pd

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler, MinMaxScaler

from keras.models import Sequential
from keras.layers import Dense, Input
from keras.callbacks import EarlyStopping
from keras.callbacks import ModelCheckpoint
```

Data science always starts with the data!

> The target variable is the median house value for California districts, expressed in $100,000's. This dataset was derived from the 1990 U.S. census, using one row per census block group. A block group is the smallest geographical unit for which the U.S. Census Bureau publishes sample data (a block group typically has a population of 600 to 3,000 people).

Dall-E's rendition of this dataset.

Source: Scikit-learn documentation.

Columns

- `MedInc` median income in block group
- `HouseAge` median house age in block group
- `AveRooms` average number of rooms per household
- `AveBedrms` average # of bedrooms per household
- `Population` block group population
- `AveOccup` average number of household members
- `Latitude` block group latitude
- `Longitude` block group longitude
- `MedHouseVal` median house value (**target**)

Source: Scikit-learn documentation.

Import the data

```
features, target = fetch_california_housing(as_frame=True, return_X_y=True)
features
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude
0	8.3252	41.0	6.984127	1.023810	322.0	2.555556	37.88	-122.23
1	8.3014	21.0	6.238137	0.971880	2401.0	2.109842	37.86	-122.22
2	7.2574	52.0	8.288136	1.073446	496.0	2.802260	37.85	-122.24
...	...	...	...	...	...	...	...	...
20637	1.7000	17.0	5.205543	1.120092	1007.0	2.325635	39.43	-121.22
20638	1.8672	18.0	5.329513	1.171920	741.0	2.123209	39.43	-121.32
20639	2.3886	16.0	5.254717	1.162264	1387.0	2.616981	39.37	-121.24

20640 rows × 8 columns

Train/validation/test split

```
1 X_main, X_test, y_main, y_test = train_test_split(
2     features, target, test_size=0.2, random_state=1)
3 X_train, X_val, y_train, y_val = train_test_split(
4     X_main, y_main, test_size=0.25, random_state=1)
5
6 num_features = features.shape[1]
7 print(X_train.shape, X_val.shape, X_test.shape)
```

```
(12384, 8) (4128, 8) (4128, 8)
```

Linear regression baseline

Refit the linear regression from earlier; we'll compare neural networks against this baseline.

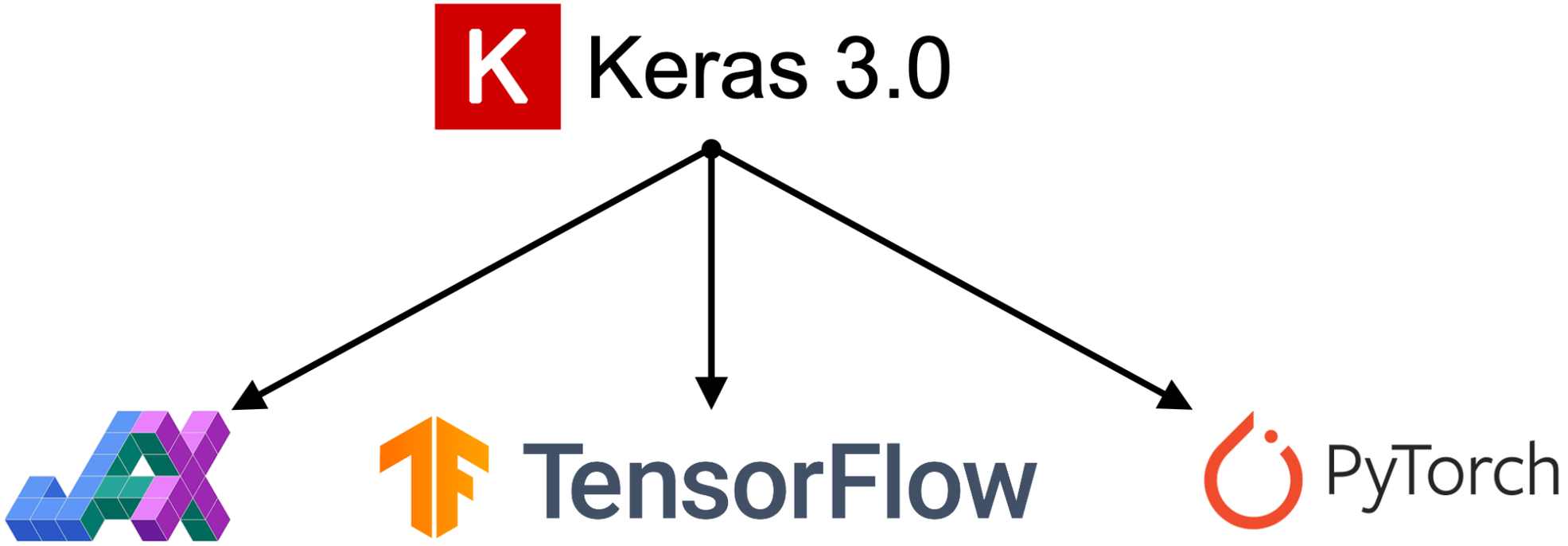
```
lr = LinearRegression()
lr.fit(X_train, y_train)

mse_lr_train = mean_squared_error(y_train, lr.predict(X_train))
mse_lr_val = mean_squared_error(y_val, lr.predict(X_val))

mse_train = {"Linear Regression": mse_lr_train}
mse_val = {"Linear Regression": mse_lr_val}
```

Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- **Our First Neural Network**
- Force Positive Predictions
- Preprocessing
- Early Stopping

What are Keras and PyTorch?

Keras is a common way of specifying, training, and using neural networks. It gives a simple interface to *various backend* libraries, including PyTorch.



The Keras application programming interface (API)

Source: Melissa Renard (2025)

Create a Keras ANN model

Decide on the architecture: a simple fully-connected network with one hidden layer with 30 neurons.

Create the model:

```
model = Sequential(
    [Input((num_features,)),
     Dense(30, activation="leaky_relu"),
     Dense(1, activation="leaky_relu")]
)
```

Inspect the model

```
1 model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 30)	270
dense_1 (Dense)	(None, 1)	31

Total params: 301 (1.18 KB)
Trainable params: 301 (1.18 KB)
Non-trainable params: 0 (0.00 B)

The model is initialised randomly

```
model = Sequential([Dense(30, activation="leaky_relu"), Dense(1, activation="leaky_relu")
model.predict(X_val.head(3), verbose=0)
```

```
array([[-139.05],
       [ -84.57],
       [  -5.82]], dtype=float32)
```

```
model = Sequential([Dense(30, activation="leaky_relu"), Dense(1, activation="leaky_relu")
model.predict(X_val.head(3), verbose=0)
```

```
array([[-108.21],
       [ -64.74],
       [  -7.1 ]], dtype=float32)
```

Controlling the randomness

```
random.seed(2026)

model = Sequential([Dense(30, activation="leaky_relu"), Dense(1, activation="leaky_relu")])

display(model.predict(X_val.head(3), verbose=0))

random.seed(2026)
model = Sequential([Dense(30, activation="leaky_relu"), Dense(1, activation="leaky_relu")])

display(model.predict(X_val.head(3), verbose=0))
```

```
array([[467.02],
       [289.13],
       [ 20.93]], dtype=float32)
```

```
array([[467.02],
       [289.13],
       [ 20.93]], dtype=float32)
```

Fit the model

```
random.seed(2026)

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="leaky_relu")
])

model.compile("adam", "mse")
%time hist = model.fit(X_train, y_train, epochs=5, verbose=0)
hist.history["loss"]
```

```
CPU times: user 1.04 s, sys: 78.3 ms, total: 1.12 s
Wall time: 1.07 s
```

```
[1095.7493896484375,
 6.3134589195251465,
 4.662435531616211,
 3.2442092895507812,
 1.773606300354004]
```

Make predictions

```
1 y_pred = model.predict(X_train[:3], verbose=0)
2 y_pred
```

```
array([[ 1.74],
       [-0.83],
       [ 1.77]], dtype=float32)
```

Note

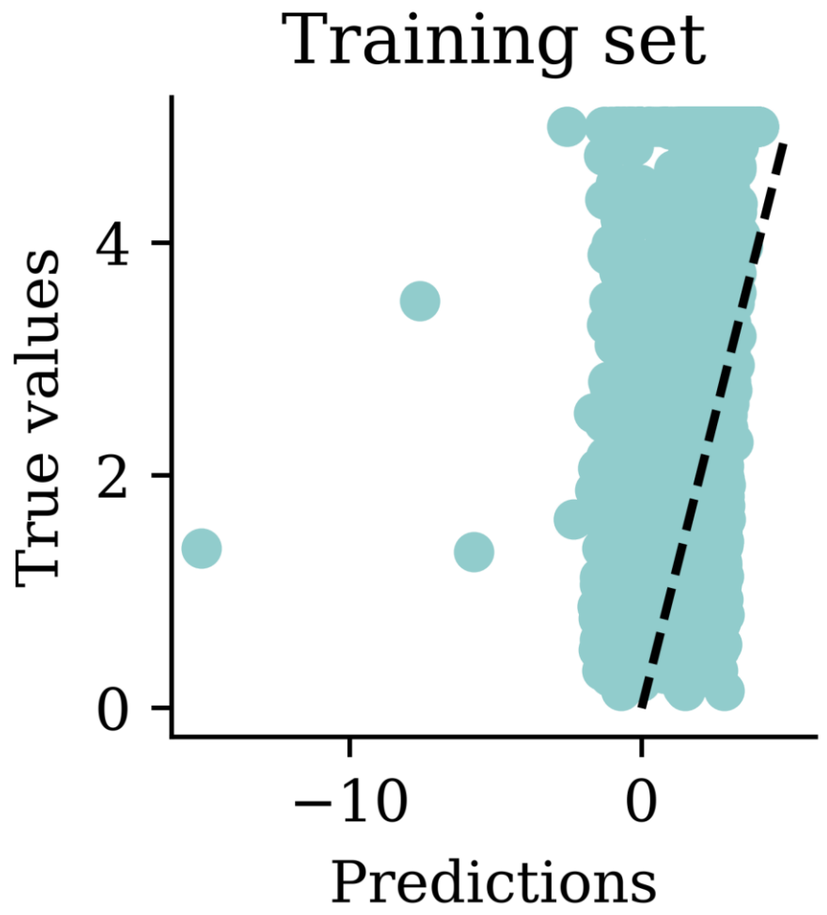
The `.predict` gives us a 'matrix' not a 'vector'. Calling `.flatten()` will convert it to a 'vector'.

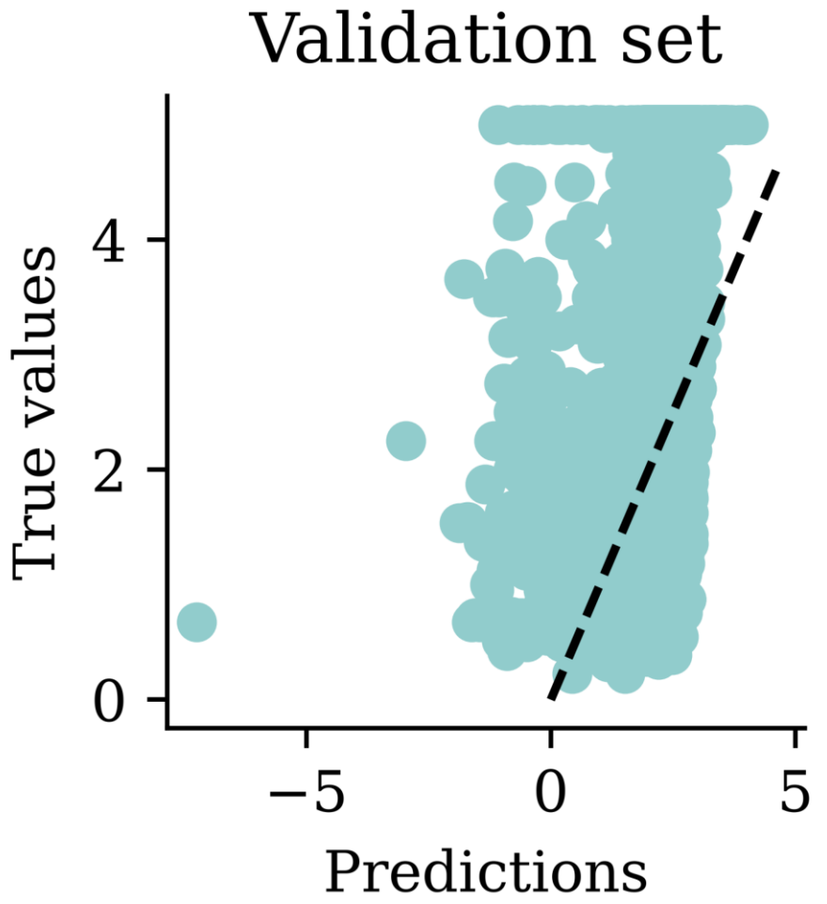
```
1 print(f"Original shape: {y_pred.shape}")
2 y_pred = y_pred.flatten()
3 print(f"Flattened shape: {y_pred.shape}")
4 y_pred
```

```
Original shape: (3, 1)
Flattened shape: (3,)
```

```
array([ 1.74, -0.83,  1.77], dtype=float32)
```

Plot the predictions





Assess the model

```
y_pred = model.predict(X_val, verbose=0)
mean_squared_error(y_val, y_pred)
```

```
1.322503585825187
```

```
mse_train["Basic ANN"] = mean_squared_error(
    y_train, model.predict(X_train, verbose=0)
)
mse_val["Basic ANN"] = mean_squared_error(y_val, model.predict(X_val, verbose=0))
```

Some predictions are negative:

```
y_pred = model.predict(X_val, verbose=0)
y_pred.min(), y_pred.max()
```

```
(np.float32(-7.2446113), np.float32(4.038248))
```

```
y_val.min(), y_val.max()
```

```
(np.float64(0.225), np.float64(5.00001))
```

Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- **Force Positive Predictions**
- Preprocessing
- Early Stopping

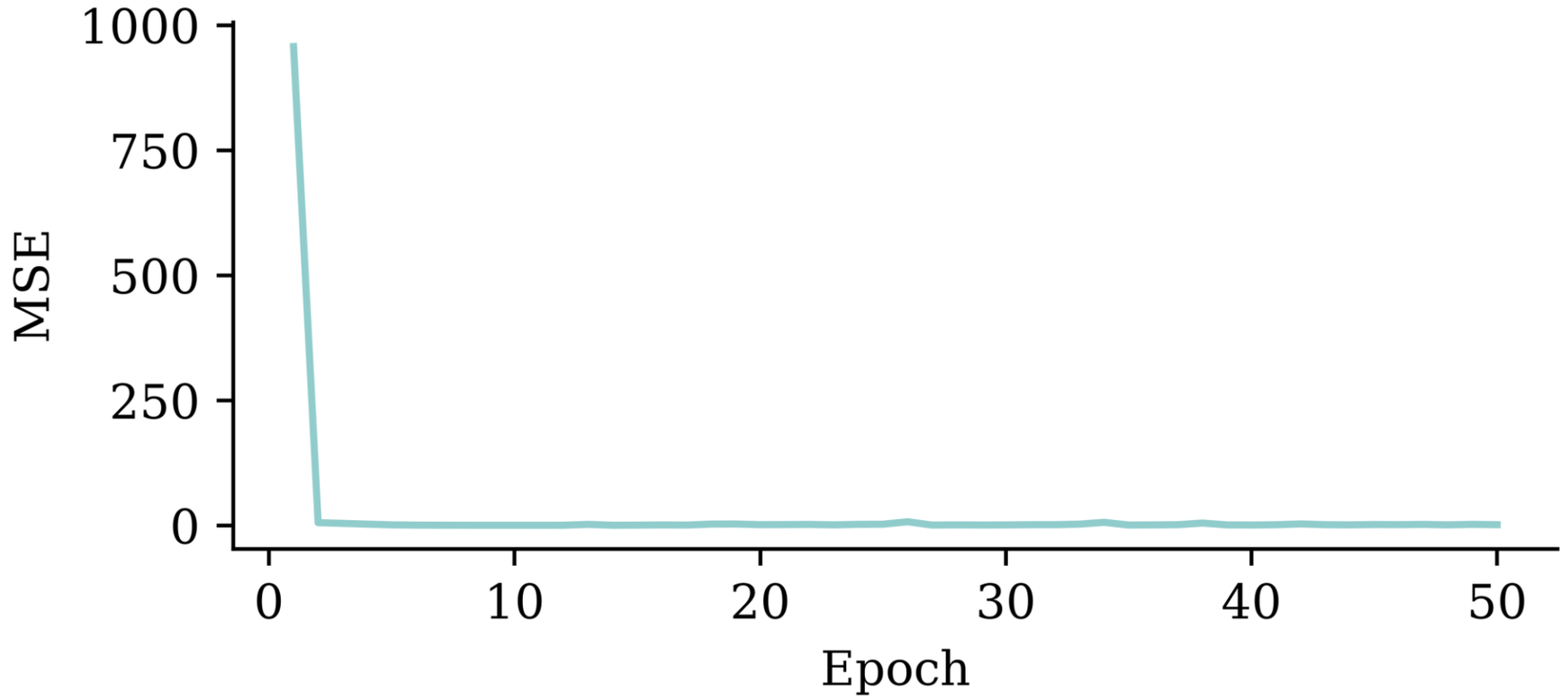
Try running for longer

```
random.seed(2026)

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="leaky_relu")
])

model.compile("adam", "mse")

%time hist = model.fit(X_train, y_train, epochs=50, verbose=0)
```

```
CPU times: user 10.1 s, sys: 704 ms, total: 10.8 s
Wall time: 10.3 s
```

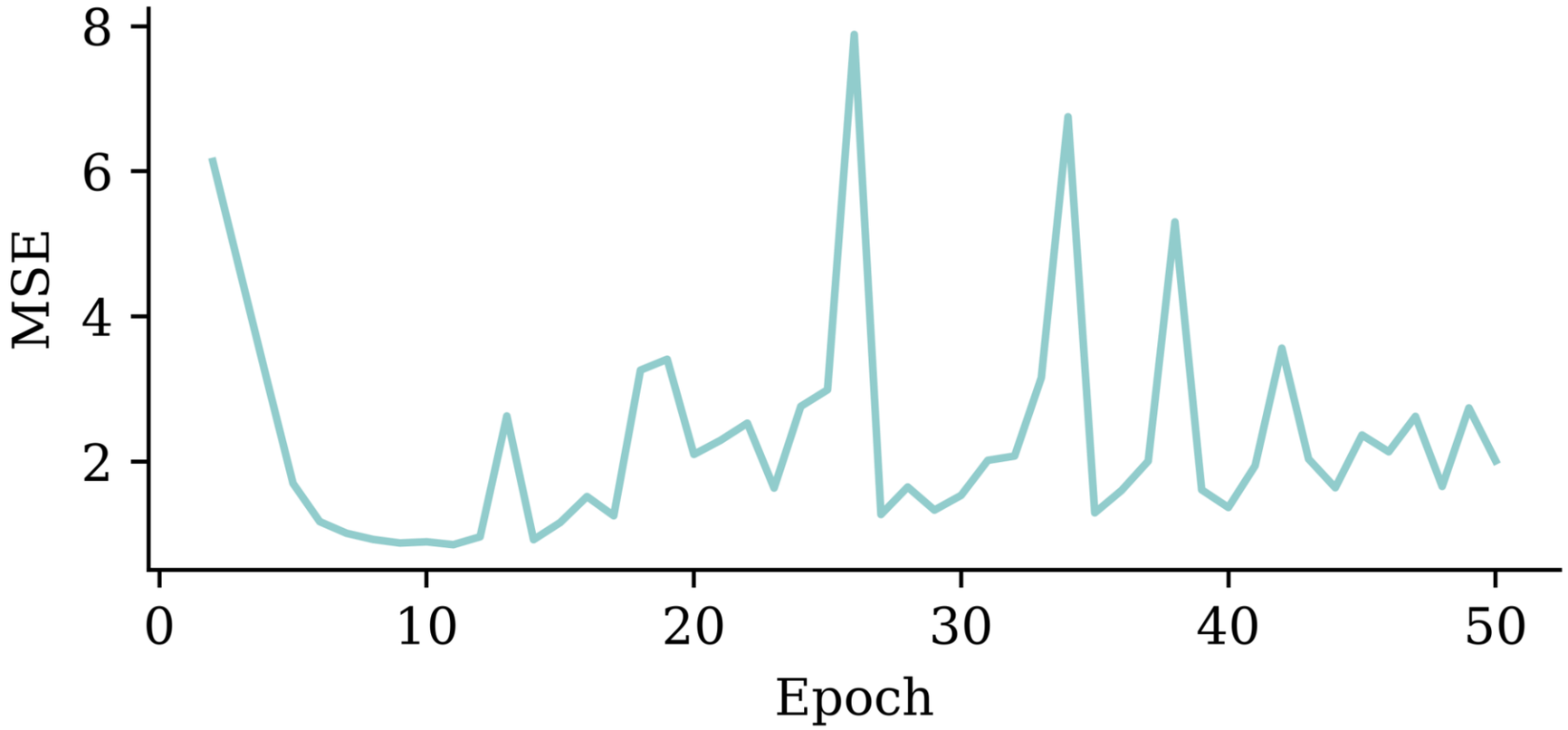
Loss curve

```
1 plt.plot(range(1, 51), hist.history["loss"])
2 plt.xlabel("Epoch")
3 plt.ylabel("MSE");
```



Loss curve

```
1 plt.plot(range(2, 51), hist.history["loss"][1:])
2 plt.xlabel("Epoch")
3 plt.ylabel("MSE");
```



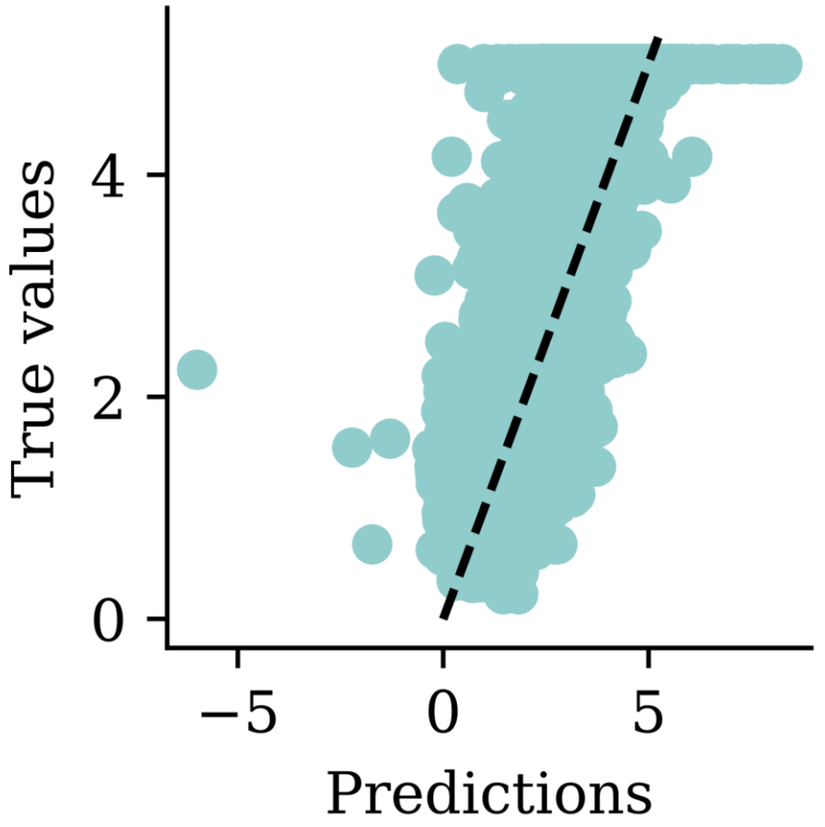
Predictions

```
y_pred = model.predict(X_val, verbose=0)
print(f"Min prediction: {y_pred.min():.2f}")
print(f"Max prediction: {y_pred.max():.2f}")
```

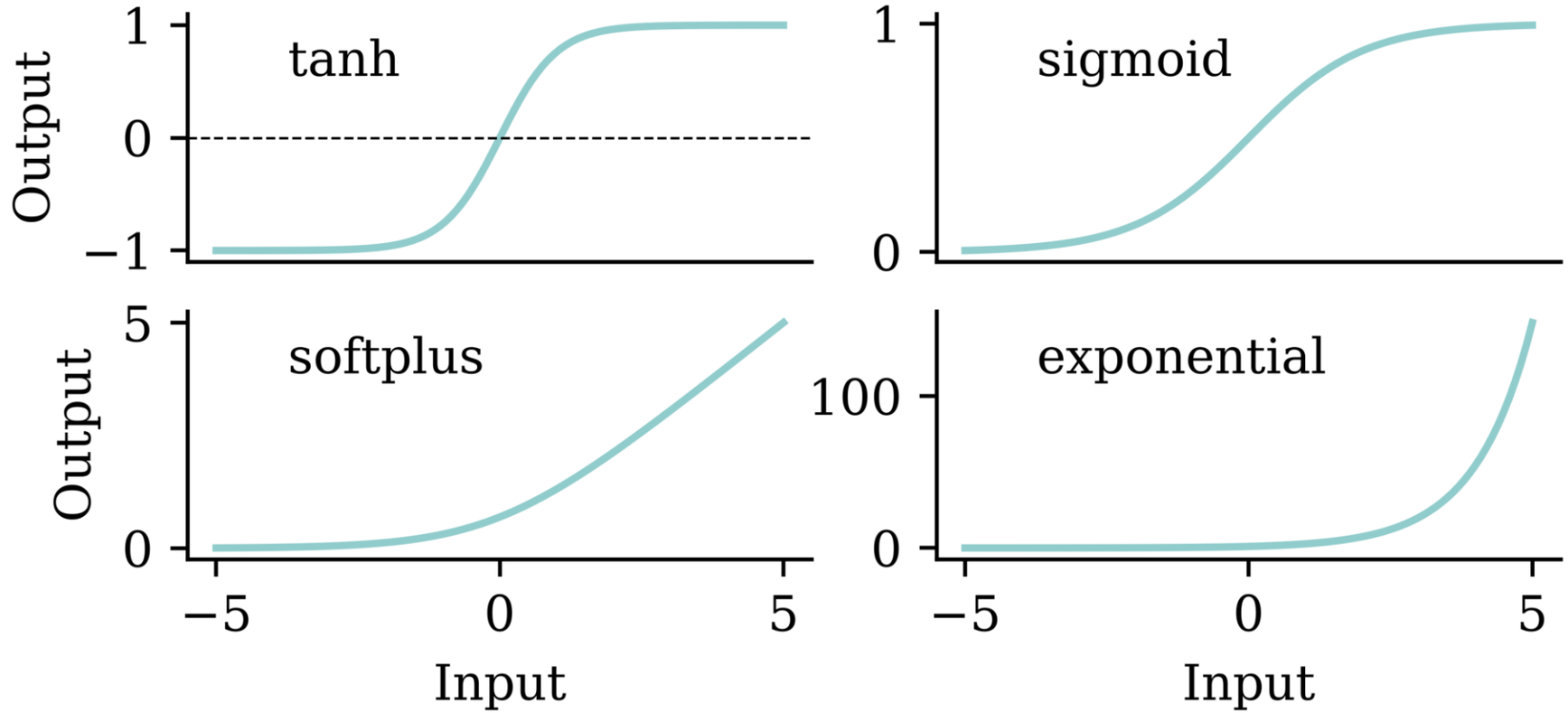
```
Min prediction: -6.00
Max prediction: 8.25
```

```
plt.scatter(y_pred, y_val)
plt.xlabel("Predictions")
plt.ylabel("True values")
add_diagonal_line()
```

```
mse_train["Long run ANN"] = mean_squared_
    y_train, model.predict(X_train, verbo
)
mse_val["Long run ANN"] = mean_squared_er
```



Try different activation functions

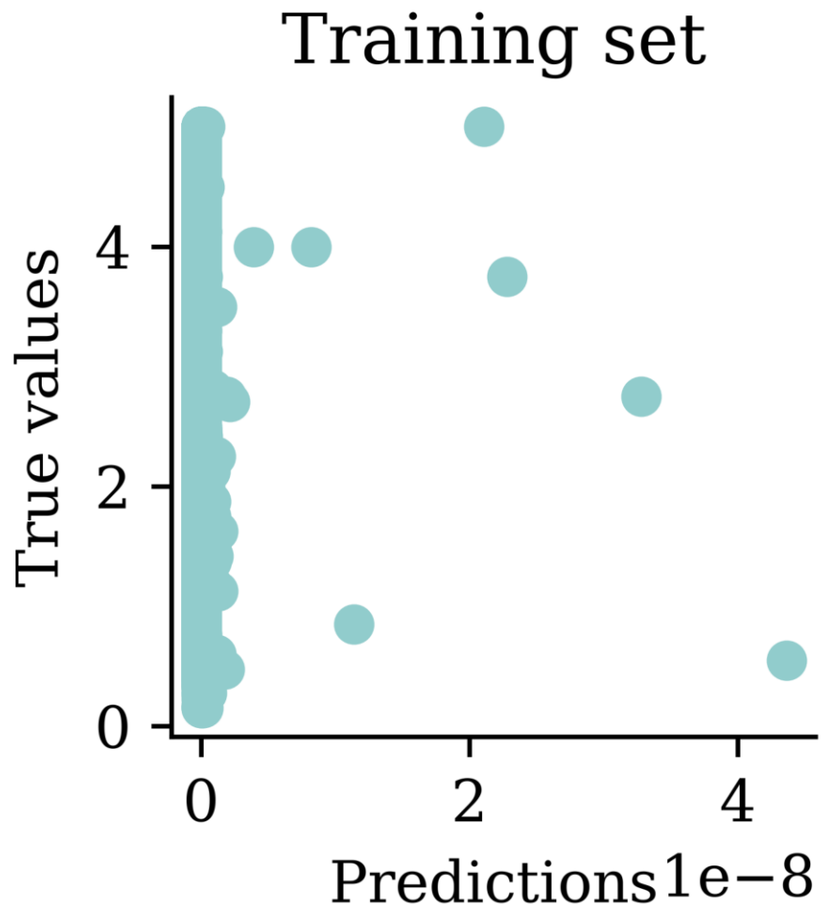


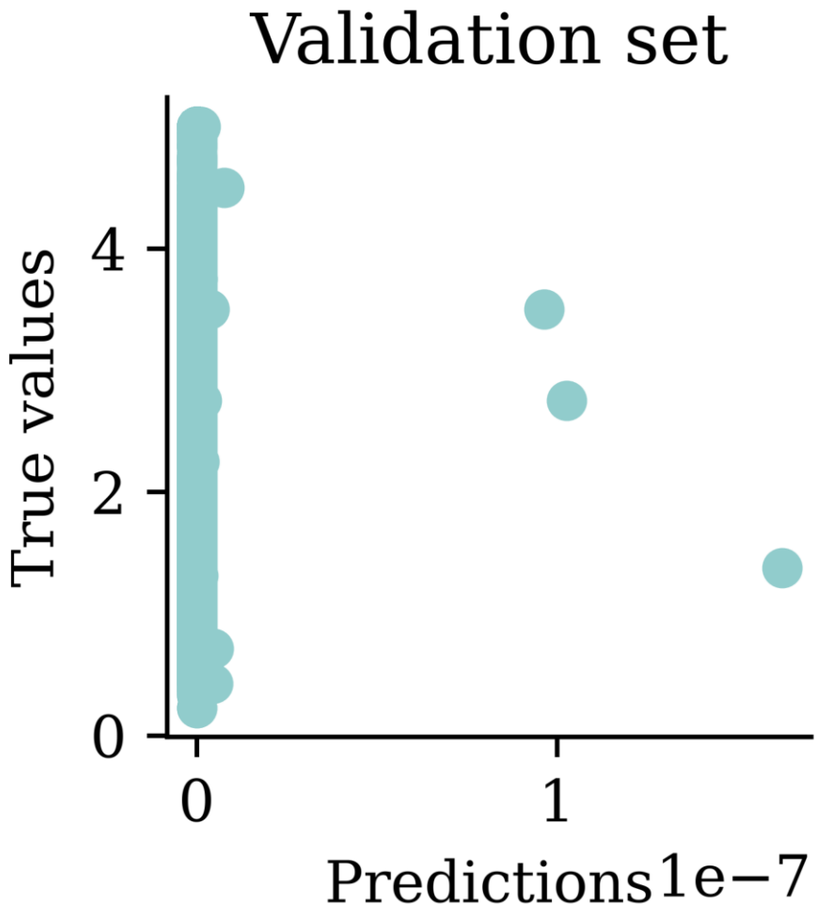
Enforce positive outputs (softplus)

```
random.seed(2026)

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="softplus")
])

model.compile("adam", "mse")

%time hist = model.fit(X_train, y_train, epochs=50, verbose=0)

losses = np.round(hist.history["loss"], 2)
print(losses[:5], "...", losses[-5:])
```

```
CPU times: user 10.1 s, sys: 736 ms, total: 10.9 s
Wall time: 10.3 s
[973.53   5.64   5.64   5.64   5.64] ... [5.64 5.64 5.64 5.64 5.64]
```

Plot the predictions





Enforce positive outputs (e^x)

```
random.seed(2026)

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="exponential")
])

model.compile("adam", "mse")

%time hist = model.fit(X_train, y_train, epochs=5, verbose=0)

losses = hist.history["loss"]
print(losses)
```

```
CPU times: user 1.01 s, sys: 70.7 ms, total: 1.08 s
Wall time: 1.03 s
[nan, nan, nan, nan, nan]
```

Same as transforming the target

4.1. Model

We fitted the following model:

$$\begin{aligned}\ln(\text{MEDIAN VALUE}) = \text{INTERCEPT} &+ \beta_2\,\text{MEDIAN INCOME} + \beta_3\,\text{MEDIAN INCOME}^2 + \beta_4\,\text{MEDIAN INCOME}^3 \\ &+ \beta_5 \ln(\text{MEDIAN(AGE)}) + \beta_6 \ln(\text{TOTAL ROOMS/POPULATION}) \\ &+ \beta_7 \ln(\text{BEDROOMS/POPULATION}) + \beta_8 \ln(\text{POPULATION/HOUSEHOLDS}) \\ &+ \beta_9 \ln(\text{HOUSEHOLDS}) \end{aligned} \tag{8}$$

The polynomial regression used by researchers who first studied this dataset.

> **Note**
>
> Fitting ln(Median Value) is mathematically identical to the `exponential` activation function in the final layer (but metrics are in different units).

Source: Pace & Barry (1997).

Good to know others results

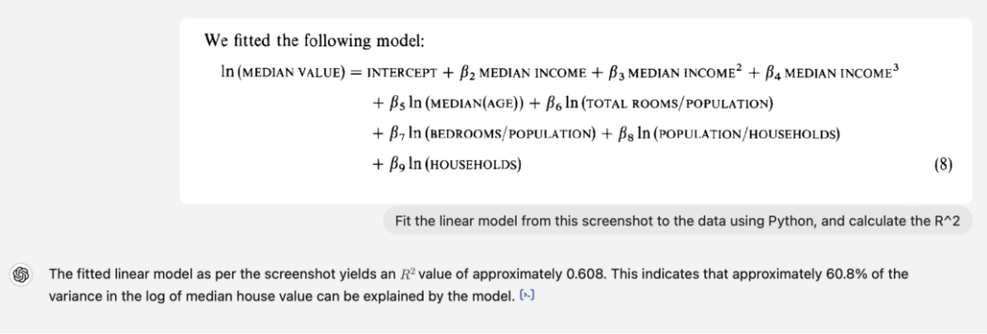
Table 1
OLS and SAR estimates for median housing prices across 20 640 California census block groups

	B_{ols}	t_{ols}	B_{sar}	t_{sar}
INTERCEPT	11.4939	275.7518	11.6637	402.5925
MEDIAN INCOME	0.4790	45.7768	0.0349	4.7104
MEDIAN INCOME^2	−0.0166	−9.4841	0.0100	8.4280
MEDIAN INCOME^3	−0.0002	−1.9157	−0.0007	−12.2444
ln(MEDIAN AGE)	0.1570	33.6123	−0.0421	−11.0942
ln(TOTAL ROOMS/POPULATION)	−0.8582	−56.1280	0.3098	24.5768
ln(BEDROOMS/POPULATION)	0.8043	38.0685	−0.1926	−11.8049
ln(POPULATION/HOUSEHOLDS)	−0.4077	−20.8762	−0.0342	−2.3582
ln(HOUSEHOLDS)	0.0477	13.0792	0.0034	1.5569
α			0.8536	
R^2	0.6078		0.8594	
Median $\|e\|$	0.2101		0.1084	
Execution time			1130 s	
Number of likelihood evaluations			10	

That basic model gets R^2 of 0.61, but their fancy model gets 0.86.

Source: Pace & Barry (1997).

GPT can double-check these results



We fitted the following model:

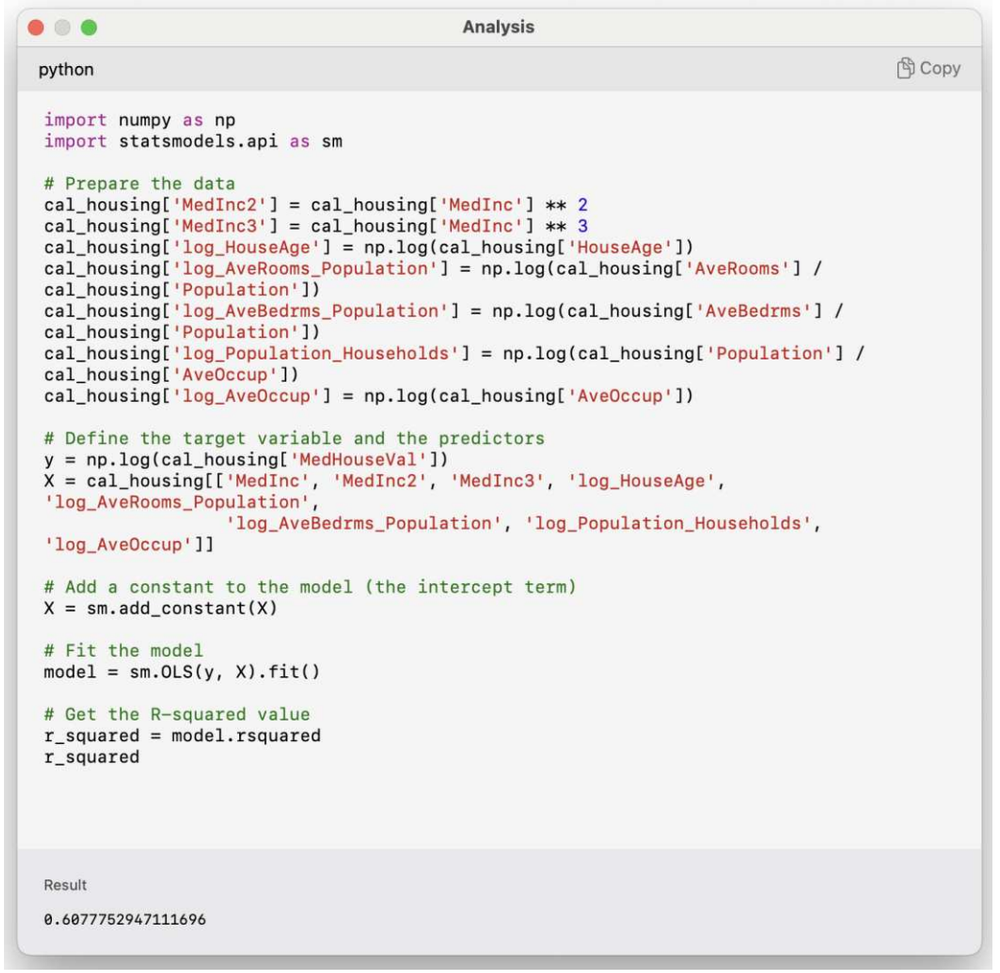
$$\ln(\text{MEDIAN VALUE}) = \text{INTERCEPT} + \beta_2\,\text{MEDIAN INCOME} + \beta_3\,\text{MEDIAN INCOME}^2 + \beta_4\,\text{MEDIAN INCOME}^3 + \beta_5 \ln(\text{MEDIAN(AGE)}) + \beta_6 \ln(\text{TOTAL ROOMS/POPULATION}) + \beta_7 \ln(\text{BEDROOMS/POPULATION}) + \beta_8 \ln(\text{POPULATION/HOUSEHOLDS}) + \beta_9 \ln(\text{HOUSEHOLDS}) \quad (8)$$

Fit the linear model from this screenshot to the data using Python, and calculate the R^2

The fitted linear model as per the screenshot yields an R^2 value of approximately 0.608. This indicates that approximately 60.8% of the variance in the log of median house value can be explained by the model.

Asking GPT to check it.

I'd previously given it the CSV of the data.



Analysis

python

Copy

```
import numpy as np
import statsmodels.api as sm

# Prepare the data
cal_housing['MedInc2'] = cal_housing['MedInc'] ** 2
cal_housing['MedInc3'] = cal_housing['MedInc'] ** 3
cal_housing['log_HouseAge'] = np.log(cal_housing['HouseAge'])
cal_housing['log_AveRooms_Population'] = np.log(cal_housing['AveRooms'] /
cal_housing['Population'])
cal_housing['log_AveBedrms_Population'] = np.log(cal_housing['AveBedrms'] /
cal_housing['Population'])
cal_housing['log_Population_Households'] = np.log(cal_housing['Population'] /
cal_housing['AveOccup'])
cal_housing['log_AveOccup'] = np.log(cal_housing['AveOccup'])

# Define the target variable and the predictors
y = np.log(cal_housing['MedHouseVal'])
X = cal_housing[['MedInc', 'MedInc2', 'MedInc3', 'log_HouseAge',
'log_AveRooms_Population',
                 'log_AveBedrms_Population', 'log_Population_Households',
'log_AveOccup']]

# Add a constant to the model (the intercept term)
X = sm.add_constant(X)

# Fit the model
model = sm.OLS(y, X).fit()

# Get the R-squared value
r_squared = model.rsquared
r_squared
```

Result

```
0.6077752947111696
```

The code it wrote & ran.

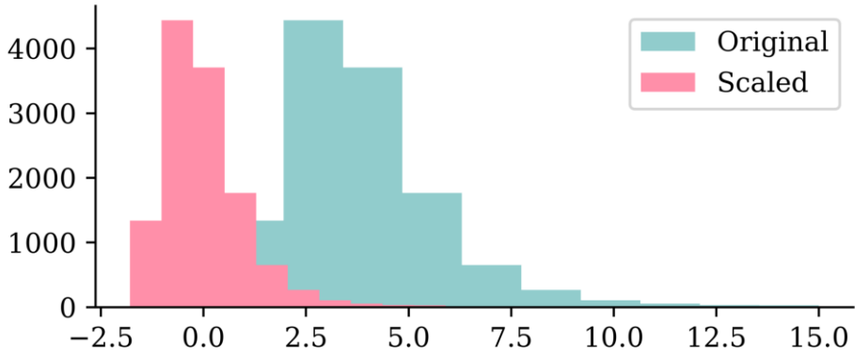
Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- **Preprocessing**
- Early Stopping

Re-scaling the inputs

```
scaler = StandardScaler()
scaler.fit(X_train)

X_train_sc = scaler.transform(X_train)
X_val_sc = scaler.transform(X_val)
X_test_sc = scaler.transform(X_test)
```

```
plt.hist(X_train.iloc[:, 0])
plt.hist(X_train_sc[:, 0])
plt.legend(["Original", "Scaled"]);
```



Same model with scaled inputs

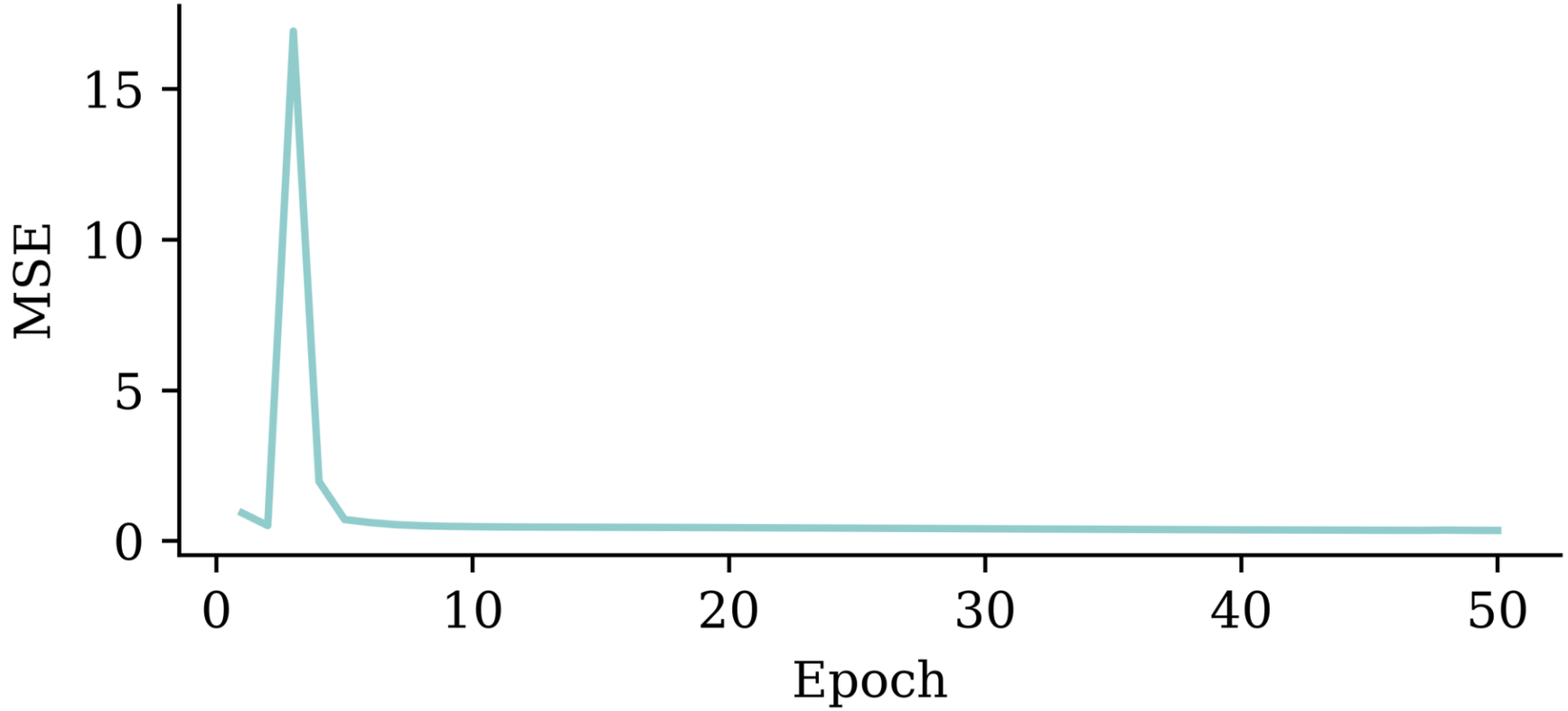
```
random.seed(2026)

model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="exponential")
])

model.compile("adam", "mse")

%time hist = model.fit(X_train_sc, y_train, epochs=50, verbose=0)
```

```
CPU times: user 9.52 s, sys: 711 ms, total: 10.2 s
Wall time: 9.71 s
```

Note the use of `X_train_sc` instead of `X_train`.

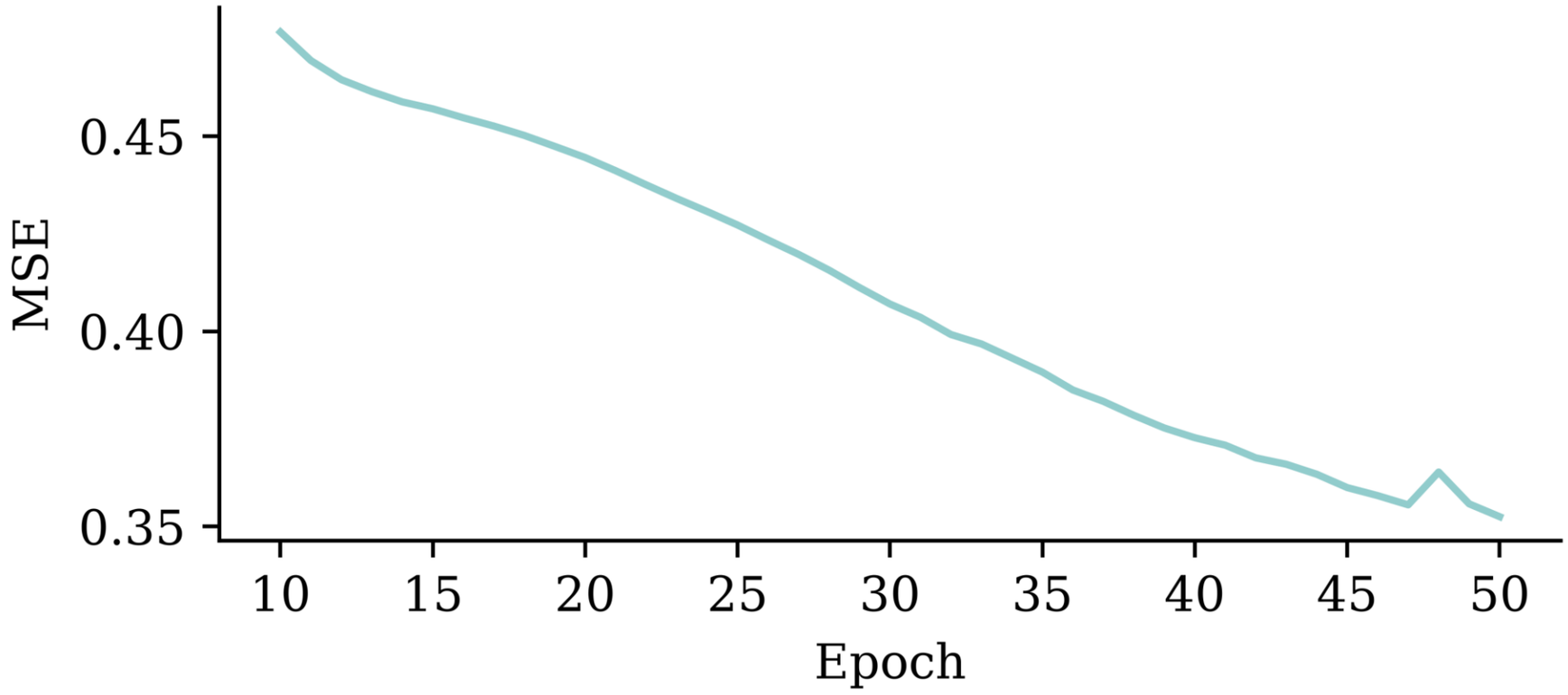
Loss curve

```
1 plt.plot(range(1, 51), hist.history["loss"])
2 plt.xlabel("Epoch")
3 plt.ylabel("MSE");
```



Loss curve

```
1 plt.plot(range(10, 51), hist.history["loss"][9:])
2 plt.xlabel("Epoch")
3 plt.ylabel("MSE");
```



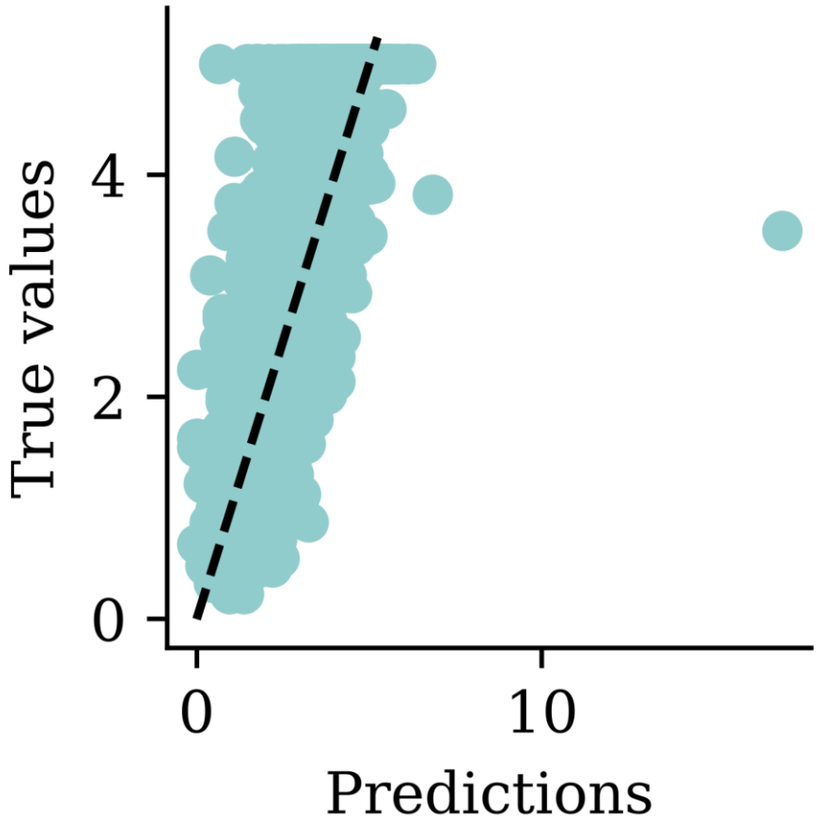
Predictions

```
y_pred = model.predict(X_val_sc, verbose=0)
print(f"Min prediction: {y_pred.min():.2f}")
print(f"Max prediction: {y_pred.max():.2f}")
```

```
Min prediction: 0.00
Max prediction: 16.96
```

```
plt.scatter(y_pred, y_val)
plt.xlabel("Predictions")
plt.ylabel("True values")
add_diagonal_line()
```

```
mse_train["Exp ANN"] = mean_squared_error
    y_train, model.predict(X_train_sc, ve
)
mse_val["Exp ANN"] = mean_squared_error(y
```



Comparing MSE (smaller is better)

On training data:

```
1 mse_train
```

```
{'Linear Regression': 0.5291948207479792,
 'Basic ANN': 1.3506622360084013,
 'Long run ANN': 0.6312395755307988,
 'Exp ANN': 0.3468037711915672 4}
```

On validation data (expect *worse*, i.e. bigger):

```
1 mse_val
```

```
{'Linear Regression': 0.5059420205381369,
 'Basic ANN': 1.322503585825187,
 'Long run ANN': 0.629845824224333,
 'Exp ANN': 0.3807926503100916 3}
```

Comparing models (train)

```
train_results = pd.DataFrame({"Model": mse_train.keys(), "MSE": mse_train.values()})
train_results.sort_values("MSE", ascending=False)
```

	Model	MSE
1	Basic ANN	1.350662
2	Long run ANN	0.631240
0	Linear Regression	0.529195
3	Exp ANN	0.346804

Comparing models (validation)

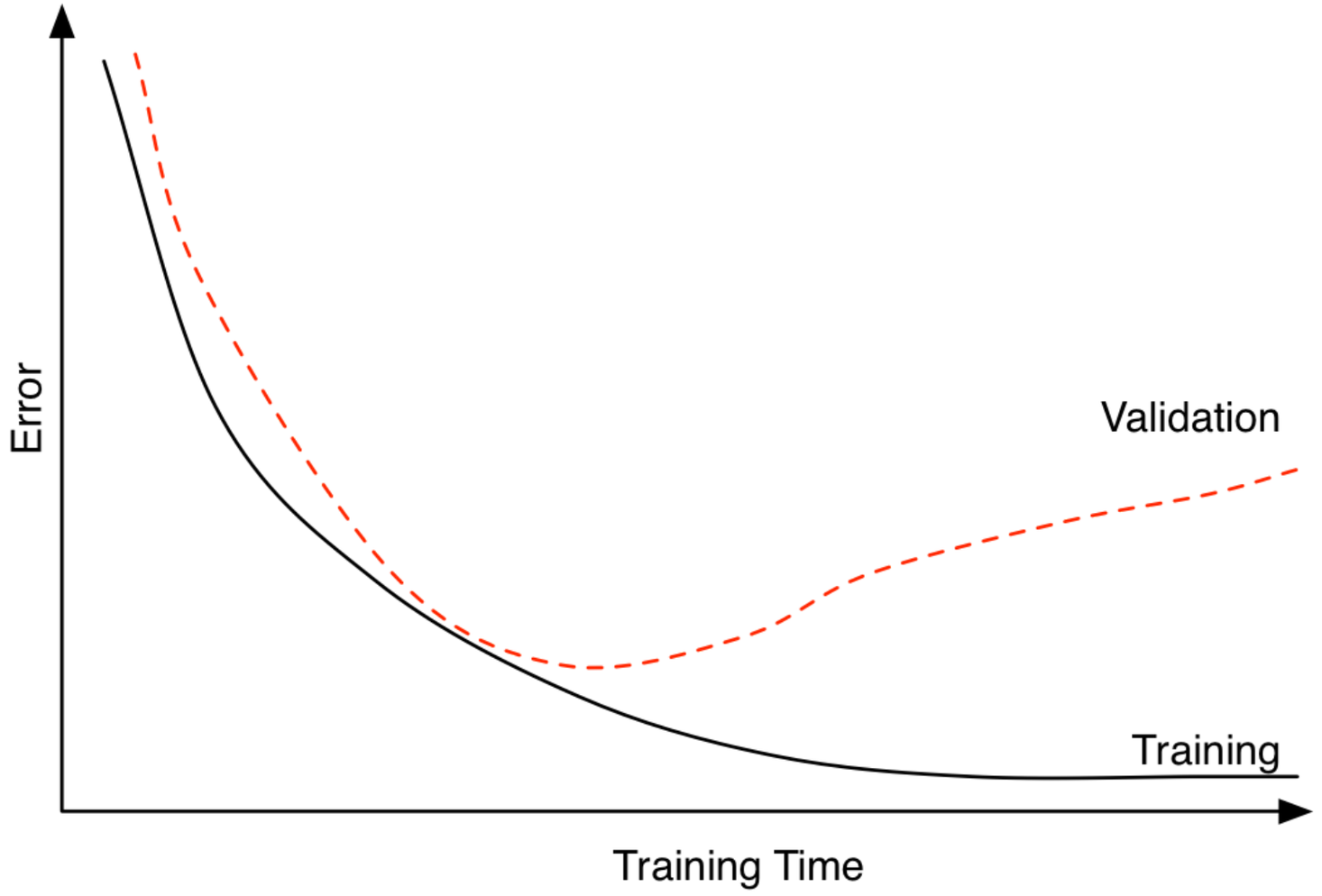
```
1 val_results = pd.DataFrame({"Model": mse_val.keys(), "MSE": mse_val.values()})
2 val_results.sort_values("MSE", ascending=False)
```

	Model	MSE
1	Basic ANN	1.322504
2	Long run ANN	0.629846
0	Linear Regression	0.505942
3	Exp ANN	0.380793

Lecture Outline

- Artificial Intelligence
- Deep Learning Successes
- Types of Machine Learning Tasks
- Neural Networks
- California House Price Prediction
- Our First Neural Network
- Force Positive Predictions
- Preprocessing
- **Early Stopping**

Choosing when to stop training



Illustrative loss curves over time.

Source: Heaton (2022), Applications of Deep Learning, Part 3.4: Early Stopping.

Try early stopping

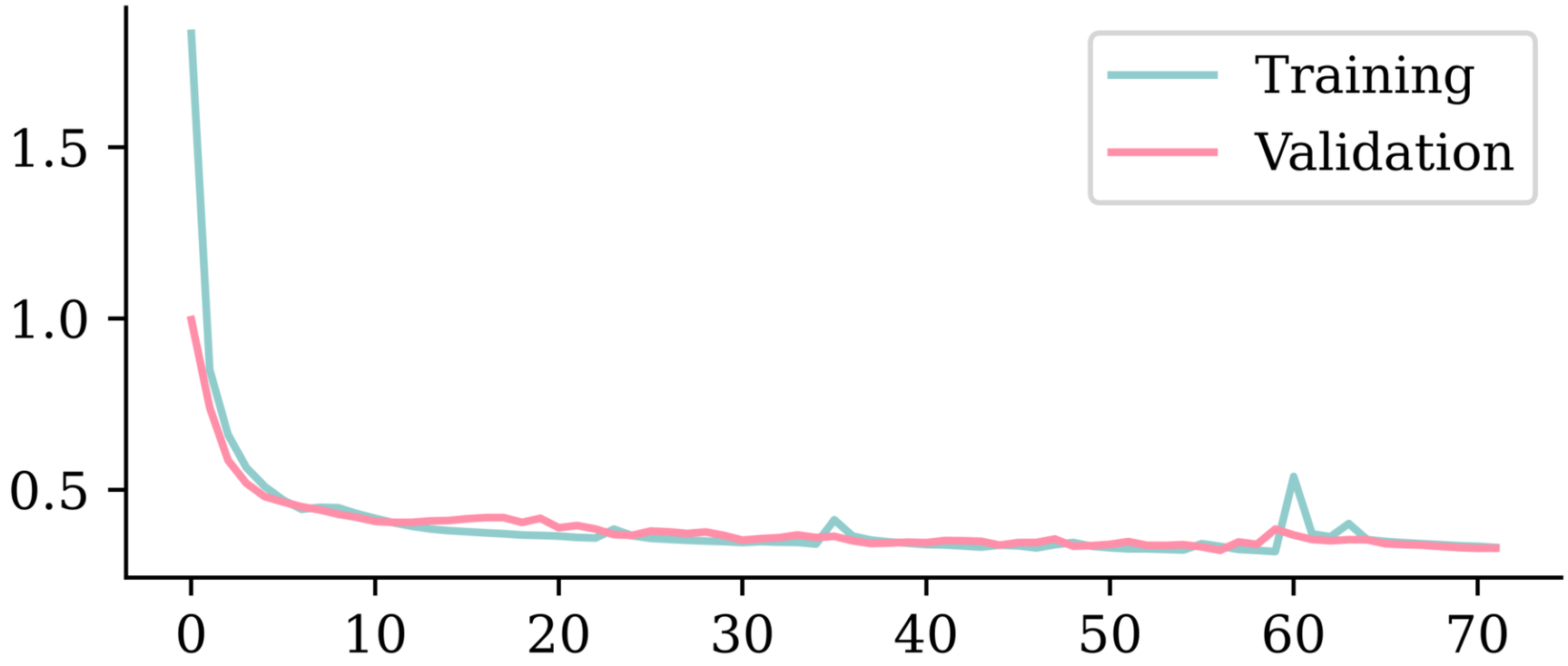
Hinton calls it a “beautiful free lunch”

```
random.seed(2026)
model = Sequential([
    Dense(30, activation="leaky_relu"),
    Dense(1, activation="exponential")
])
model.compile("adam", "mse")

es = EarlyStopping(restore_best_weights=True, patience=15)

%time hist = model.fit(X_train_sc, y_train, epochs=1_000, \
    callbacks=[es], validation_data=(X_val_sc, y_val), verbose=0)
print(f"Keeping model at epoch #{len(hist.history['loss'])-15}.")
```

```
CPU times: user 17.3 s, sys: 1.42 s, total: 18.7 s
Wall time: 17.7 s
Keeping model at epoch #57.
```

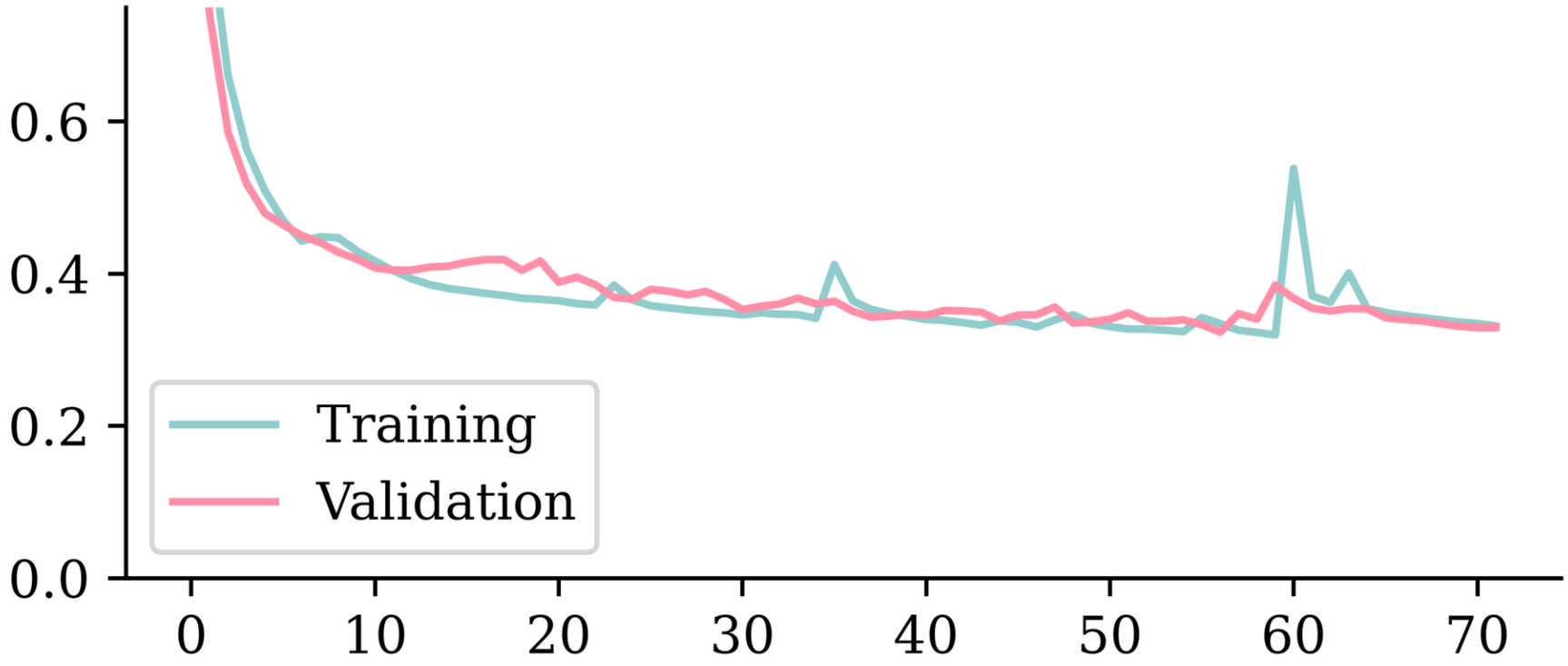
Loss curve

```
1 plt.plot(hist.history["loss"])
2 plt.plot(hist.history["val_loss"])
3 plt.legend(["Training", "Validation"]);
```

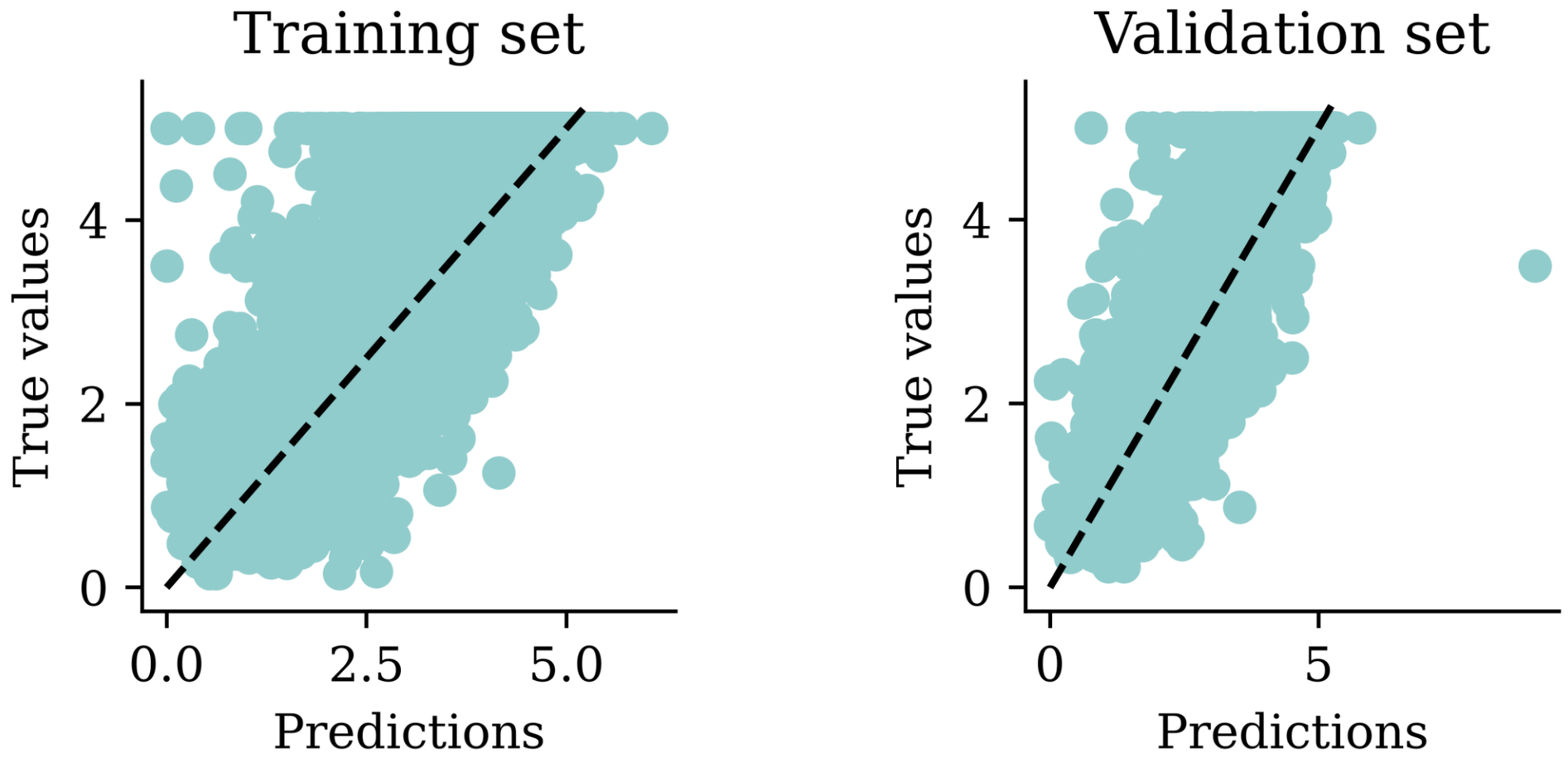


Loss curve II

```
1 plt.plot(hist.history["loss"])
2 plt.plot(hist.history["val_loss"])
3 plt.ylim([0, 0.75])
4 plt.legend(["Training", "Validation"]);
```



Predictions



Comparing models (validation)

	Model	MSE
1	Basic ANN	1.322504
2	Long run ANN	0.629846
0	Linear Regression	0.505942
3	Exp ANN	0.380793
4	Early stop ANN	0.323157

The test set

Evaluate *only the final/selected model* on the test set.

```
1 mean_squared_error(y_test, model.predict(X_test_sc, verbose=0))
```

```
0.33164115325909604
```

```
1 model.evaluate(X_test_sc, y_test, verbose=0)
```

```
0.3316410779953003
```

Keras model methods

- `compile`: specify the loss function and optimiser
- `fit`: learn the parameters of the model
- `predict`: apply the model
- `evaluate`: apply the model and calculate a metric

```
random.seed(12)
model = Sequential()
model.add(Dense(1, activation="relu"))
model.compile("adam", "poisson")
model.fit(X_train, y_train, verbose=0)
y_pred = model.predict(X_val, verbose=0)
print(model.evaluate(X_val, y_val, verbose=0))
```

```
4.4610700607299805
```

Package Versions

```
1 from watermark import watermark
2 print(watermark(python=True, packages="keras,matplotlib,numpy,pandas,seaborn,scipy,torch"
```

```
Python implementation: CPython
Python version       : 3.14.5
IPython version      : 9.15.0

keras     : 3.15.0
matplotlib: 3.11.0
numpy     : 2.5.0
pandas    : 3.0.3
seaborn   : 0.13.2
scipy     : 1.18.0
torch     : 2.12.1
```

Glossary

- activations, activation function
- artificial neural network
- biases (in neurons)
- callbacks
- classification problem
- cost/loss function
- deep network, network depth
- dense or fully-connected layer
- early stopping
- epoch
- feed-forward neural network
- hidden layer
- Keras, TensorFlow, PyTorch
- labelled/unlabelled data
- machine learning
- minimax algorithm
- neural network architecture
- perceptron
- ReLU
- representation learning
- sigmoid activation function
- targets
- training/validation/test split
- weights (in a neuron)

References

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, *27*.

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, *33*, 6840–6851.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, *25*, 1097–1105.

Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.

Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, *33*(3), 291–297.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, *3*(3), 210–229.